MALLA REDDY UNIVERSITY

A Major Project Phase-1 Project Report

CodeGalaxy: An Agentic AI Based Interactive Programming Learning Platform

Design & developed by

HARSH KUMAR	**2311CS040064**
ABHISHEK PRATAP KUSHWAHA	**2311CS040003**
ADITYA RAJ	**2311CS040006**

GUIDED BY

Mrs. G Divya Jyothi
Assistant Professor

Department of Cybersecurity

B. Tech IV YR - I SEM

Malla Reddy University, Hyderabad

October - 2026



Department of Computer Science & Engineering (Cyber Security)

CERTIFICATE

This is to certify that the project report entitled **"CodeGalaxy: An Agentic AI Based Interactive Programming Learning Platform", submitted by Harsh Kumar (2311CS040064), Abhishek Pratap Kushwaha (2311CS040003), Aditya Raj (2311CS040006)** towards the partial fulfillment for the award of Bachelor's Degree in Cyber Security from the Department of Cybersecurity, Malla Reddy University, Hyderabad, is a record of bonafide work done by Harsh Kumar, Abhishek Pratap Kushwaha and Aditya Raj. The results embodied in the work are not submitted to any other University or Institute for the award of any degree or diploma.

Internal Guide
Mrs. G Divya Jyothi
Assistant Professor

Dean (Cyber Security)
Dr. G. Anand Kumar
Professor & Dean

External Examiner

DECLARATION

We hereby declare that the project report "CodeGalaxy: An Agentic AI Based Interactive Programming Learning Platform" has been carried out by us and this work has been submitted to the Department of Cybersecurity Malla Reddy University, Hyderabad. We further declare that this project work has not been submitted in full or part for the award of any other degree in any other educational institutions.

Place: Hyderabad
Date:

Harsh Kumar **2311CS040064**

Abhishek Pratap Kushwaha **2311CS040003**

Aditya Raj **2311CS040006**

ACKNOWLEDGEMENT

We extend our sincere gratitude to all those who have contributed to the completion of this project report. Firstly, we would like to extend our gratitude to **Dr. V. S. K. Reddy, Vice Chancellor**, for his visionary leadership and unwavering commitment to academic excellence.

We are also grateful to **Dr. G. Anand Kumar, Dean, Department of Cyber Security, ECE & IoT** for providing us with the necessary resources and facilities to carry out this project. We extend our gratitude to our **PRC - Convenor, Dr. G. Latha**, for giving valuable inputs and timely guidelines to improve the quality of our project through a critical review process.

We would also like to express our deepest appreciation to our project guide **Mrs. G. Divya Jyothi, Assistant Professor** whose invaluable guidance, insightful feedback and unwavering support have been instrumental throughout the course of this project for successful outcomes.

We thank our project coordinator, **Mr. A. Ramesh Khanna**, Assistant Professor for his timely support.

We are deeply indebted to all of them for their support, encouragement and guidance, without which this project would not have been possible.

Harsh Kumar	**2311CS040064**
Abhishek Pratap Kushwaha	**2311CS040003**
Aditya Raj	**2311CS040006**

ABSTRACT

CodeGalaxy is an interactive, gamified and AI-driven programming learning platform developed to help beginners understand how programs actually behave at run time. Traditional programming education relies on textbooks, lectures and online judges. These help learners with syntax and final output, but they rarely explain why a program produced a result or what went wrong inside it. Concepts such as loops, arrays, linked lists, recursion, sorting and memory behaviour remain abstract, so many beginners memorise code instead of building a real mental model.

To address this gap, CodeGalaxy combines real code execution with Agentic AI assistance. The platform provides a Monaco-based workspace for Python, JavaScript, Java, C and C++, backed by a local multi-language execution engine. When a run fails, the source code, language and error output are routed to a multi-provider large-language-model (LLM) orchestrator that tries Google Gemini first and automatically falls back to Groq and then to a locally hosted Ollama model. The orchestrator returns a structured, step-by-step diagnosis instead of simply handing back a corrected program, so that the learner still has to think.

The Agentic AI layer is organised around four cooperating agent roles: a Code Analysis Agent that detects errors and suggests fixes, an Execution Visualization Agent realised through hand-built algorithm and data-structure visualisers, a Learning Assistant Agent that provides conversational help and document or image question answering through a zero-infrastructure TF-IDF retrieval-augmented-generation (RAG) pipeline, and an Assessment Agent that profiles skills, generates adaptive quizzes and challenges, and runs an AI technical-interview simulator. A planner-executor-synthesizer agent loop and a prompt-chaining module are implemented in the backend and form the basis for deeper autonomy.

Learning is made engaging through gamification: experience points, levels, streaks, badges, a leaderboard and a “solar-system” skill map whose prerequisite-linked nodes unlock as the learner progresses. A dynamic “Nebula Forge” challenge generator creates new problems on demand. The system is built as a MERN-style full-stack application (React 18 with TypeScript, Node.js with Express 5, MongoDB Atlas) with about 28,700 lines of code, 40 REST endpoints across nine backend modules and 30 frontend pages. This report documents the system as it exists in source, including its current implementation status and the engineering work still required to reach fully autonomous agent behaviour.

INDEX

CONTENTS	*Page No.*
Chapter 1 Introduction	***1 - 3***
1.1 Problem Definition & Description	*2*
1.2 Objectives of the Project	*2 - 3*
1.3 Scope of the Project	*3*
Chapter 2 System Analysis	***4 - 11***
2.1 Existing System	*4*
2.1.1 Background & Literature Survey	*4 - 5*
2.1.2 Limitations of Existing System	*5*
2.2 Proposed System	*5 - 6*
2.2.1 Advantages of Proposed System	*6 - 7*
2.2.2 Functional and Non-Functional Requirements	*7 - 8*
2.3 Software & Hardware Requirements	*8*
2.3.1 Software Requirements	*8 - 9*
2.3.2 Hardware Requirements	*9*
2.4 Feasibility Study	*9 - 11*
Chapter 3 Architectural Design	***12 - 25***
3.1 Modules Design	*12 - 13*
3.2 Methods & Algorithms Design	*13 - 14*
3.3 Project Architecture	*14*
3.3.1 Architectural Diagram	*14 - 17*
3.3.2 Data Flow Diagram	*17 - 18*
3.3.3 Sequence Diagram	*18 - 19*
3.3.4 Activity Diagram	*19*
3.3.5 Use Case Diagram	*20*
3.3.6 ER Diagram	*20 - 21*
3.3.7 REST API Design	*21 - 22*
3.3.8 Database Design (Data Dictionary)	*22 - 23*
3.4 Agentic AI Layer Design	*23 - 24*
3.5 Security & Reliability Design	*24 - 25*

Chapter 4 Application Code ***26 - 33***
4.1 Coding Blocks (Frontend & Backend Code) *26*
4.1.1 Code Execution Engine (backend/utils/executeCode.js) *26 - 27*
4.1.2 Multi-Provider LLM Router (backend/services/geminiService.js) *27*
4.1.3 AI Repair Diagnosis (aiController.js and geminiService.js) *27 - 28*
4.1.4 Planner-Executor-Synthesizer Agent (backend/services/agentService.js) *28*
4.1.5 TF-IDF Retrieval for Document Q&A (backend/services/ragService.js) *28 - 29*
4.1.6 Gamification Data Model (backend/models/UserProgress.js) *29*
4.1.7 Authentication (User.js and authMiddleware.js) *29 - 30*
4.1.8 AI Routes and Server Bootstrap (aiRoutes.js, server.js) *30*
4.1.9 Nebula Forge Challenge Generator (aiController.js) *30 - 31*
4.1.10 Algorithm Visualizer Step Generation (AlgorithmVisualizer.tsx) *31*
4.1.11 Frontend Repair Agent Call (frontend/src/components/editor/Terminal.tsx) *31*
4.2 Execution Flow *32*
4.3 Configuration, Build and Deployment *32 - 33*

Chapter 5 Testing and Results ***34 - 37***
5.1 Resulting Screens *34*
5.1.1 CodeGalaxy Home Page *34*
5.1.2 Mission Control Dashboard *34*
5.1.3 Challenge Workspace *34 - 35*
5.1.4 Algorithm Visualizer *35*
5.1.5 AI Conversational Assistant *35 - 36*
5.2 Result Analysis *36*
5.2.1 Result Summary *37*

Chapter 6 Conclusion & Future Scope ***38 - 39***
6.1 Conclusion *38*
6.2 Future Scope *38 - 39*

References ***40***

Paper Draft & Poster

LIST OF FIGURES

FIGURE NO.	*NAME OF THE FIGURE*	*PAGE NO.*
Fig.1	*System Architecture of CodeGalaxy*	*15*
Fig.2	*End-to-End Learner Workflow*	*16*
Fig.3	*Multi-Provider LLM Fallback Chain*	*17*
Fig.4	*Data Flow Diagram of CodeGalaxy*	*17*
Fig.5	*Sequence Diagram of a Code Run with AI Repair*	*18*
Fig.6	*Activity Diagram of a Challenge Submission*	*19*
Fig.7	*Use Case Diagram of CodeGalaxy*	*20*
Fig.8	*Entity-Relationship Diagram of CodeGalaxy*	*20*
Fig.9	*CodeGalaxy Home Page*	*34*
Fig.10	*Mission Control Dashboard*	*34*
Fig.11	*Challenge Workspace with Editor, Visualizer and Console*	*35*
Fig.12	*Interactive DSA Algorithm Visualizer*	*35*
Fig.13	*AI Conversational Assistant*	*36*

CHAPTER - 1
INTRODUCTION

Programming has become one of the most important skills in today's digital world. It underpins software development, web development, cybersecurity, data science, artificial intelligence and mobile application development. However, many beginners find programming difficult because most learning methods concentrate on writing code and obtaining a final output, without explaining how the code works internally.

Students often struggle with concepts such as loops, arrays, recursion, linked lists, stacks and algorithms. Most coding platforms only display the output of a program or a pass/fail verdict, which makes it hard for learners to understand how variables change, how loops iterate and how data structures evolve during execution. When a program fails, the learner usually receives a raw compiler message or stack trace and has very little guidance on what to do next.

CodeGalaxy is an Agentic AI based interactive programming learning platform designed to close this gap. It lets learners write real code in a browser workspace, execute it on a backend execution engine, and observe the result through live visualisations and AI-generated guidance. When a run fails, the platform does not just print the error: an AI orchestrator analyses the learner's source code together with the actual error output and returns a structured, step-by-step diagnosis that explains the cause without giving away the finished solution.

The platform supports five programming languages: Python, JavaScript, Java, C and C++. Learners can practise in the language they prefer and compare how the same logic is expressed in different syntaxes. The interface is intentionally simple: a challenge workspace with the problem brief, a real-time visualiser, the code editor and a console, together with an AI assistant that can be asked questions by text or voice.

Around this execution core, CodeGalaxy adds a gamified "solar-system" skill map with experience points (XP), levels, streaks and badges; an on-demand challenge generator called Nebula Forge; an AI technical-interview simulator with adaptive quizzes, generated coding problems and a mock interview chat; and a document and image question-answering feature that lets a learner upload a PDF, a text file or a screenshot and ask questions grounded in it.

By combining real execution, visualisation, Agentic AI assistance and gamification, CodeGalaxy aims to make programming education more effective, engaging and easy to understand, whether the user is a first-year student learning the basics or an advanced learner preparing for technical interviews.

1.1 Problem Definition & Description

Many beginners find it difficult to understand programming because traditional teaching focuses on syntax and final output instead of the internal behaviour of programs. Existing tools solve only part of this problem. Four specific gaps motivated this project:

- **No visibility into run-time state.** Online judges return a pass/fail verdict or a stack trace, not an explanation of why the output was produced.
- **Isolated AI help.** General-purpose chat assistants are not tied to an execution sandbox, a progress model or a curriculum, so the help they give is disconnected from what the learner actually ran.
- **Execution-blind learning management.** Classroom systems track grades and completion but never run or reason about the learner's code.
- **Spoiler answers.** Rewriting the learner's code teaches less than guiding the learner through a diagnosis of the real error.

Therefore, there is a need for a platform that connects real code execution, AI reasoning over that specific execution, visual explanation and a persistent gamified progress model in a single environment. CodeGalaxy is built to provide exactly that.

1.2 Objectives of the Project

The main objective of CodeGalaxy is to improve the learning experience of programming for beginners through real execution, visualisation, Agentic AI guidance and gamification. The specific objectives are:

- **Execute and explain code.** Provide a multi-language workspace that compiles and runs learner code and reports stdout, stderr and execution time.
- **Diagnose instead of spoil.** Use an AI orchestrator to turn failing runs into structured, step-by-step diagnoses that guide the learner without revealing the full fixed program.
- **Make concepts visible.** Animate sorting algorithms, linked lists and trees with playback controls so that abstract behaviour becomes observable.
- **Stay available at all times.** Route AI requests through a Gemini, Groq and Ollama fallback chain, with a mock generator as a last resort, so that the platform keeps working when any single provider fails.
- **Personalise the journey.** Assess the learner's skills with an AI pre-assessment, generate a prerequisite-linked learning path and give remedial explanations when a quiz is failed.

- **Keep learners motivated.** Award XP, levels, streaks and badges, and present progress through a dashboard and a solar-system skill map.
- **Prepare for careers.** Offer an AI technical-interview simulator with adaptive quizzes, live-graded coding problems and voice-enabled mock interviews.
- **Ground answers in learner material.** Let learners ask questions about their own PDFs, text files and screenshots using a retrieval-augmented-generation pipeline.

1.3 Scope of the Project

The scope of CodeGalaxy is to provide an interactive, AI-assisted and gamified platform for learning programming concepts. It is aimed mainly at beginners and intermediate learners, but it is also useful to teachers, who can monitor class progress through a teacher dashboard, and to self-learners preparing for technical interviews.

Functionally, the project covers the following areas: user registration and JWT-based authentication; an AI pre-assessment and generated learning path; a Monaco-based code workspace for Python, JavaScript, Java, C and C++; local code execution with time and output limits; AI error repair, code analysis and a conversational assistant; interactive algorithm and data-structure visualisation; the Nebula Forge challenge generator; document and image question answering; the gamification engine with a leaderboard; the AI interview-prep module; and learner and teacher analytics.

The topics covered include variables, loops, conditions, functions, arrays, linked lists, binary trees, recursion, searching and sorting algorithms (bubble, selection, insertion, quick and merge sort) and basic data structures.

Some capabilities are deliberately outside the current scope and are listed as future work in Chapter 6: container-level isolation of executed code, a semantic vector-search upgrade of the retrieval pipeline, wiring the planner-executor-synthesizer agent loop into the live repair endpoint, token streaming over Server-Sent Events, per-tenant rate limiting, an offline-first mobile experience, collaborative coding and online competitions.

CHAPTER - 2
SYSTEM ANALYSIS

2.1 Existing System

The existing system of programming education depends mainly on textbooks, classroom lectures, handwritten notes and basic coding exercises. These methods focus on learning syntax and producing the correct output rather than on understanding the logic behind a program. Students are often taught to reproduce programs, but they do not see what the interpreter or compiler does with each statement.

Three broad categories of tool are available to a self-taught or classroom learner today. Online judges in the style of HackerRank, LeetCode and Codeforces execute submissions against hidden test cases and report pass or fail, but they give no visibility into intermediate program state and little explanation of a wrong answer. General-purpose AI coding assistants will explain or even rewrite a learner's code on request, but they are not integrated with an execution sandbox, a progress model or a curriculum. Classroom learning-management systems track completion and grades but do not execute or reason about code at all.

Another limitation of the existing system is the lack of engagement. Programming practice can become repetitive when there are few interactive features that reward regular effort. Many students lose interest and struggle to build confidence.

2.1.1 Background & Literature Survey

Programming education has become increasingly important because of the rapid growth of technology and the demand for skilled software professionals. Research has long shown that learning to program is difficult, largely because many topics are abstract and require logical thinking [1]. Traditional lectures and exercises mainly emphasise syntax and output rather than how programs execute internally, which leaves conceptual gaps.

Program visualisation has repeatedly been shown to help. Studies of visualisation in programming courses report that showing step-by-step execution improves understanding of loops, memory and data structures [2]. Python Tutor lets learners watch variable changes, stack frames and control flow during execution [3], while VisuAlgo provides animations of sorting, searching and graph algorithms [4]. Visual program simulation has also been proposed as a way to teach the notional machine behind introductory programming [9].

Block-based environments such as Scratch remove syntax errors and let beginners focus on logic [5]. Gamification, defined as the use of game design elements in non-game contexts [6],

has been found to raise motivation and engagement, and game-based platforms such as CodeCombat apply it to coding practice [7]. Broader reviews of computer science education [8] and of learning and teaching programming [10] confirm the importance of active, feedback-rich practice.

Recent advances in large language models have created new possibilities. Retrieval-augmented generation (RAG) grounds a model's answers in retrieved documents and reduces unsupported answers [11]. Agentic approaches interleave reasoning and acting, or decompose a task into a plan that is executed step by step [12]. Classical term-weighting methods such as TF-IDF remain a simple and effective way to rank text passages without any vector database [13].

Although many tools provide either visualisation or gamification or AI help, very few combine real execution, AI reasoning over the actual error output, visualisation and a persistent gamified progress model in one environment. Several also support only a single language or depend on a single AI provider. This gap is what CodeGalaxy addresses.

2.1.2 Limitations of Existing System

The main limitations of the existing approaches can be summarised as follows:

- **No run-time insight.** Learners cannot see how variables change, how loops execute or how a recursive call stack grows and unwinds. They must trace code manually, which is slow and error-prone.
- **Generic error feedback.** Compilers and judges report raw messages or a failing test case, with no guided explanation of the underlying mistake.
- **Disconnected AI help.** Chat assistants do not see the learner's execution result, progress or curriculum, and often rewrite the code, which encourages copying instead of learning.
- **Single-provider fragility.** Tools that depend on one AI service stop working when its key expires, its quota is exhausted or its model is retired.
- **Low engagement.** Few platforms reward regular practice with levels, streaks, badges and visible progress.
- **Limited career support.** Interview preparation is usually separate from the learning platform and not adapted to the learner's weak topics.

2.2 Proposed System

The proposed system, CodeGalaxy, is an interactive, gamified and Agentic AI based platform for programming education. A learner writes code in a browser workspace, runs it on the

backend execution engine and receives output, visualisations and AI guidance in one place. If the run fails, the source code, language and actual error output are sent to the AI orchestrator, which returns a structured diagnosis that explains the cause and the steps to fix it without giving the full answer.

The system is organised around four cooperating agent roles. The Code Analysis Agent detects syntax and logic errors and suggests improvements. The Execution Visualization Agent presents execution through animated visualisers for sorting, linked lists and trees. The Learning Assistant Agent answers questions, streams chat replies and answers questions about uploaded documents and images. The Assessment Agent profiles skills, generates adaptive quizzes and challenges, and evaluates coding solutions and interview answers.

All AI features run through one provider-agnostic router that tries Google Gemini, then Groq, then a local Ollama model, and finally a hand-written mock generator. The platform therefore degrades gracefully instead of failing. Gamification (XP, levels, streaks, badges and a solar-system skill map) and an analytics dashboard keep learners motivated and informed, while a teacher dashboard gives instructors a view of class progress.

Table 2.1 compares typical existing tools with CodeGalaxy on the capabilities that matter most to a beginner.

Capability	Online judges	Chat assistants	Visualisation tools	CodeGalaxy
Real code execution	Yes	No	Partial	Yes (5 languages)
Run-time visual explanation	No	No	Yes	Yes
AI diagnosis tied to the actual error	No	Generic	No	Yes
Avoids spoiling the solution	Not applicable	Rarely	Not applicable	Yes, by design
Gamified progress and skill map	Limited	No	No	Yes
Resilient multi-provider AI	Not applicable	No	Not applicable	Yes
Interview preparation	Partial	Partial	No	Yes

2.2.1 Advantages of Proposed System

- **Guided diagnosis instead of spoilers.** Failing runs are explained step by step, which teaches debugging rather than copying.

- **Execution-aware AI.** AI reasoning is based on the actual source, language and stderr of the learner's run, not on a generic question.
- **Visual understanding.** Algorithm and data-structure visualisers with playback controls make abstract behaviour observable.
- **High availability.** The Gemini, Groq, Ollama and mock fallback chain has already absorbed model-deprecation incidents without an outage.
- **Multi-language practice.** Python, JavaScript, Java, C and C++ are supported, so learners can compare syntax across languages.
- **Personalised learning path.** An AI pre-assessment sets the starting point and a failed quiz triggers a remedial explanation before the next node unlocks.
- **Motivation through gamification.** XP, levels, streaks, badges, a leaderboard and a solar-system map reward consistent practice.
- **Career readiness.** The interview simulator offers adaptive quizzes, graded coding problems and voice-enabled mock interviews.
- **Grounded document Q&A.** Learners can ask questions about their own notes and screenshots, and the assistant is instructed to say when the context is insufficient.
- **Low operating cost.** The retrieval pipeline needs no vector database, and a local Ollama model allows offline operation without paid APIs.

Overall, the proposed system provides a more effective alternative to traditional programming education by combining real execution, visualisation, Agentic AI guidance, gamification and multi-language support.

2.2.2 Functional and Non-Functional Requirements

The functional requirements describe what CodeGalaxy must do for its users:

1. The system shall allow users to register, log in or use social sign-in and shall maintain a secure JWT session.
2. The system shall assess a learner's skills with an AI questionnaire and generate a prerequisite-linked learning path.
3. The system shall let users write and run code in Python, JavaScript, Java, C and C++.
4. The system shall return output, error text and execution time, and shall treat only real failures as errors.
5. The system shall diagnose failed runs through the AI layer and return ordered steps without the full solution.

6. The system shall provide co-pilot analysis, hints and complexity feedback on the active code.

7. The system shall visualise sorting, searching, linked-list and tree operations with playback controls.

8. The system shall generate new challenges from a plain-English prompt.

9. The system shall answer questions about uploaded PDFs, text files and images. 10. The system shall award XP, levels, streaks and badges and show a leaderboard. 11. The system shall provide adaptive interview quizzes, graded coding problems and a mock interview.

12. The system shall give teachers an analytics view of class progress.

The non-functional requirements are summarised in Table 2.2.

Table 2.2 Non-functional requirements of CodeGalaxy

Attribute	Requirement
Performance	Code runs are limited to 8 seconds and 10 MB of output; chat replies can be streamed to the client.
Availability	AI features must keep working when a provider fails, using the Gemini, Groq, Ollama and mock fallback chain.
Security	Passwords are hashed with bcrypt; private routes require a valid JWT; admin and teacher routes check roles.
Usability	Responsive interface, dark space theme, clear console output and voice input for the assistant.
Scalability	Stateless REST API that can run on several instances; the in-memory RAG store is a known single-instance limit.
Maintainability	Modular routes, controllers, services and models; one shared AI router for all providers.
Portability	Runs on Windows, Linux and macOS with Node.js and the language toolchains installed.

2.3 Software & Hardware Requirements

2.3.1 Software Requirements

Component	Specification
Operating system	Windows, Linux or macOS
Frontend	React 18, TypeScript, Vite, Tailwind CSS, Radix UI, Framer Motion
Editor and visuals	Monaco Editor, React Flow (@xyflow/react), Recharts
Backend	Node.js (v18 or higher), Express 5, JWT, bcryptjs, multer, pdf-parse
Database	MongoDB Atlas with Mongoose 9; in-memory document store for RAG

Component	Specification
AI providers	Google Gemini, Groq (LPU inference), Ollama (local models)
Language toolchains	Python 3, Node.js, JDK (javac/java), gcc, g++
IDE / tools	VS Code, Git, MongoDB Compass
Browser	Google Chrome, Mozilla Firefox, Microsoft Edge
Hosting	Vercel (frontend), Render (backend)

2.3.2 Hardware Requirements

Component	Specification
Processor	Intel i3 or higher multi-core processor
RAM	Minimum 4 GB (8 GB recommended when running a local Ollama model)
Storage	Minimum 10 GB free storage space
Internet connection	Required for Gemini, Groq and MongoDB Atlas; optional when only Ollama is used
Display	Standard monitor with good resolution
Input devices	Keyboard and mouse; microphone and speakers for voice features

2.4 Feasibility Study

The feasibility study shows that CodeGalaxy is technically, economically, operationally, legally and commercially feasible. It can be built with widely available open-source technologies, runs on modest hardware and is simple to use for students and teachers.

1. Technical Feasibility: The platform is a standard web application built with React, TypeScript, Node.js, Express and MongoDB, all of which are mature and well documented. The code editor uses Monaco, the visualisers are React components, and code execution uses the compilers and interpreters already available on the host. The AI layer uses the official Gemini SDK, Groq's OpenAI-compatible API and Ollama's local HTTP API. The prototype itself, with a working execution engine, a three-provider AI router and 30 pages, demonstrates technical feasibility. Developers need knowledge of front-end and back-end development, REST APIs, database design, and prompt design for LLMs.

2. Economic Feasibility: Development uses free and open-source tools, and the team supplies its own development effort. Hosting can use free or low-cost tiers of Vercel, Render and MongoDB Atlas. AI cost is controlled by using the free tiers of Gemini and Groq with a local Ollama model as a no-cost fallback. Ongoing costs are limited to maintenance, API usage at

scale and hosting upgrades. Revenue options include premium learning features, subscriptions and partnerships with institutions and training centres.

3. Operational Feasibility: The interface is designed so that beginners can write code, run it and read the guidance without training. Reliability is supported by the fallback chain and mock generators, which keep every AI feature usable even if all providers fail. Teachers can use the analytics dashboard to monitor student progress. For many simultaneous users, the backend will need rate limiting and horizontally scalable storage, which are listed as future work.

4. Legal Feasibility: User data such as credentials, progress and scores must be protected. Passwords are hashed with bcrypt and sessions use signed JSON Web Tokens. Any third-party problems, learning material or datasets must be used with proper permission, and AI provider terms of service must be followed. Uploaded documents are held only in server memory for the session.

5. Market Feasibility: There is a growing demand for programming skills among students, professionals and institutions. Because most existing tools provide either output-only execution or generic chat help, a platform that combines real execution, visual explanation, guided AI diagnosis, gamification and interview preparation addresses a clear need in schools, colleges, coaching centres and among self-learners.

6. Time Feasibility: The project was developed incrementally using existing technologies. Core features such as authentication, the code workspace and execution engine, and the dashboard were completed first, followed by the AI router, visualisers, gamification, retrieval pipeline and interview simulator. Advanced items such as sandbox isolation and semantic retrieval can be added in later iterations without redesigning the system.

Table 2.3 summarises the outcome of the feasibility study.

Table 2.3 Feasibility summary.

Feasibility type	Assessment	Key evidence
Technical	High	Mature stack; working prototype with execution engine, AI router and 30 pages
Economic	High	Open-source tools, free-tier hosting and APIs, local Ollama fallback at no cost
Operational	High	Simple interface, graceful degradation, teacher analytics dashboard
Legal	Medium to high	Hashed passwords and JWT; permissions needed for third-party content and API terms

Feasibility type	Assessment	Key evidence
Market	High	Demand for programming skills; few tools combine execution, AI diagnosis and gamification
Time	High	Incremental delivery; advanced items such as sandboxing deferred to later iterations

CHAPTER - 3
ARCHITECTURAL DESIGN

3.1 Modules Design

The system is implemented using the following modules:

1. **User Profile & Authentication Module:** Handles sign-up, login and social sign-in. Passwords are hashed with bcryptjs and a signed JWT protects every private route. The profile stores learning preferences, per-topic skill levels and the active learning path.

2. **Code Editor Module:** A Monaco-based editor for Python, JavaScript, Java, C and C++ with a console panel that shows output, execution status, time and memory.

3. **Execution Engine Module:** Writes submitted code to a uniquely named temporary file, compiles or interprets it with the language toolchain, applies an 8-second timeout and a 10 MB output cap, and returns output, success flag and time.

4. **AI Orchestration Module (Agentic AI):** A provider-agnostic router that sends every AI request to Gemini, then Groq, then Ollama, with a mock fallback. It hosts the code-analysis, repair, chat, assessment, challenge-generation and interview functions, together with the planner-executor-synthesizer agent loop.

5. **Visualization Module:** Hand-built React components that animate sorting algorithms, linked lists and binary trees, with step controls, speed control, custom input and live complexity information.

6. **Learning Path & Assessment Module:** Converts the AI pre-assessment into a skill map, generates a 5 to 10 node learning path of prerequisite-linked skill nodes and applies remedial help when a quiz is failed.

7. **Challenge & Nebula Forge Module:** Serves curated challenges and generates new ones from a plain-English prompt, including title, Markdown description, starter template, test cases, difficulty and XP reward.

8. **RAG Document & Image Q&A Module:** Parses uploaded PDFs and text files, chunks the text, ranks chunks with TF-IDF cosine similarity and answers with a grounded prompt. Images are answered through a vision-capable model.

9. **Gamification Module:** Maintains XP, level, streak, completed challenges and unlocked badges, and exposes progress, badge and leaderboard endpoints.

10. **Interview Prep Module:** Generates adaptive quizzes, coding problems and mock interview turns, evaluates submitted solutions and supports voice input and output.

11. **Analytics Module:** Provides learner charts (XP velocity, performance matrix, knowledge-core donut) and a teacher view of class progress.

12. **Database Module:** MongoDB Atlas collections for users, progress, challenges, learning paths, skill nodes, badges and interview-prep state.

3.2 Methods & Algorithms Design

CodeGalaxy combines classical algorithms with LLM orchestration methods. The principal methods are described below.

1. Execution and Failure Classification. The submitted source is written to a temporary file and run through the correct toolchain. A non-zero exit code, a timeout kill or a spawn failure is treated as failure. Output on stderr alone is deliberately not treated as failure because compilers routinely print warnings for programs that ran correctly. Stdout and stderr are both returned so that the console and the AI repair agent can use them.

2. Multi-Provider Routing with Fallback. Every AI call goes through a single choke point. In automatic mode the router calls Gemini; on any error such as an expired key, exhausted quota, network failure or retired model it calls Groq; if that fails it calls a local Ollama model; and if every provider fails or returns unparsable output, a keyword-matched mock generator returns structurally valid content. A start-up diagnostic makes one live call to each provider so that misconfiguration is visible immediately.

3. Guided Diagnosis (Code Analysis Agent). The repair prompt contains the learner's code, the language, the exact error and the test-case context. The model is instructed to return strict JSON with a diagnosis, ordered steps and a tiny conceptual snippet, but not the solved program. The response is parsed with a tolerant JSON extractor that also handles responses wrapped in code fences.

4. Planner-Executor-Synthesizer Agent Loop. The agent first asks the LLM for a 3 to 6 step plan, then executes each step in order while feeding all earlier step results into the next prompt, and finally asks for one consolidated deliverable. Because later steps see earlier outputs, the sequence behaves as a coherent reasoning chain rather than independent calls.

5. TF-IDF Retrieval. Uploaded text is cleaned and split into chunks of about 900 characters with 150 characters of overlap, breaking on a sentence boundary where possible. A TF-IDF vector is built for each chunk using inverse document frequency $\log((N+1)/(df+1))+1$. A question is vectorised in the same way, chunks are ranked by cosine similarity and the top four

are placed in a bounded context block. If no chunk scores above zero the first chunks are used so that the model is never left without context.

6. Adaptive Remediation. When a learner fails a module quiz, the system requests a simplified explanation of the failed concept instead of unlocking the next skill node. A passed quiz unlocks the next node of the learning path.

7. Visualization Algorithms. Each visualiser first generates a list of states and then plays them back with controls for play, pause, step and speed. For bubble sort every individual comparison and swap is recorded as a state that marks the comparing, swapping and sorted positions. The visualiser also offers selection, insertion, quick and merge sort through the same player, where the current implementation shows progressive sorted-state steps, and finer per-comparison steps for these algorithms are listed as future work. Linear and binary search, linked-list and tree operations use the same state-list approach.

8. Gamification Method. Passing a challenge awards its XP reward, updates the level and streak, and checks badge rules. Progress is stored in a dedicated progress collection that is separate from the authentication record.

3.3 Project Architecture

3.3.1 Architectural Diagram

CodeGalaxy is organised into four layers, shown in Fig. 1: a React 18 and TypeScript frontend; a Node.js and Express 5 API server; an AI-orchestration layer made of backend service modules for Gemini, Groq and Ollama; and a data layer that combines MongoDB Atlas with an in-memory document store used by the RAG feature. Every arrow in the diagram corresponds to an HTTP call or an in-process function call in the source code.

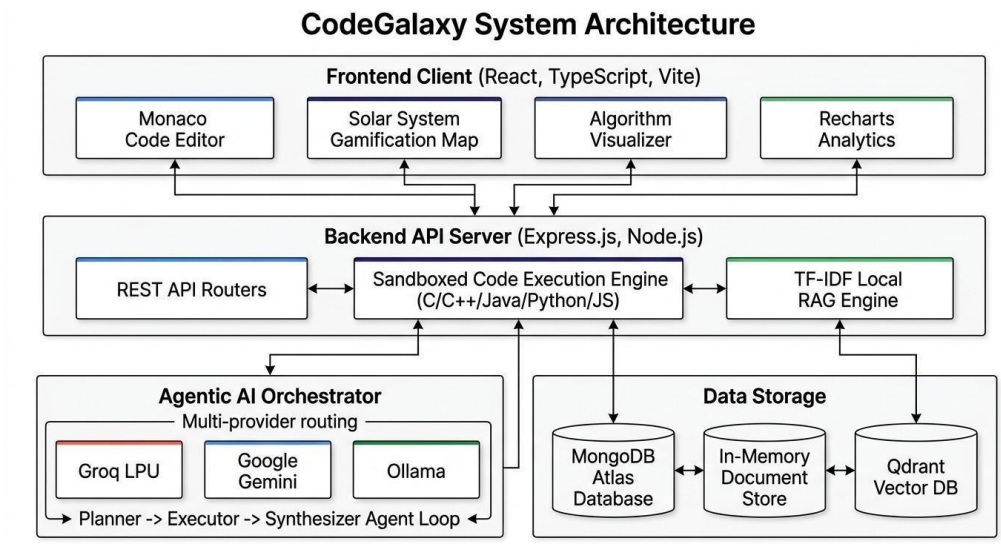


Fig. 1 System Architecture of CodeGalaxy

1. Frontend Client. Provides the Monaco code editor, the solar-system gamification map, the algorithm visualiser and the Recharts analytics. It communicates with the backend through REST calls that carry a JWT bearer token.

2. Backend API Server. Express routers expose nine route modules. The code-execution engine and the TF-IDF RAG engine run inside the same server process.

3. Agentic AI Orchestrator. Routes each request through the multi-provider chain (Gemini, Groq, Ollama) and hosts the planner-executor-synthesizer agent loop.

4. Data Storage. MongoDB Atlas stores durable data. The in-memory document store holds uploaded documents for the RAG feature. A Qdrant vector store is scaffolded for a future semantic-search upgrade.

The end-to-end operation of the platform from the learner's point of view is shown in Fig. 2.

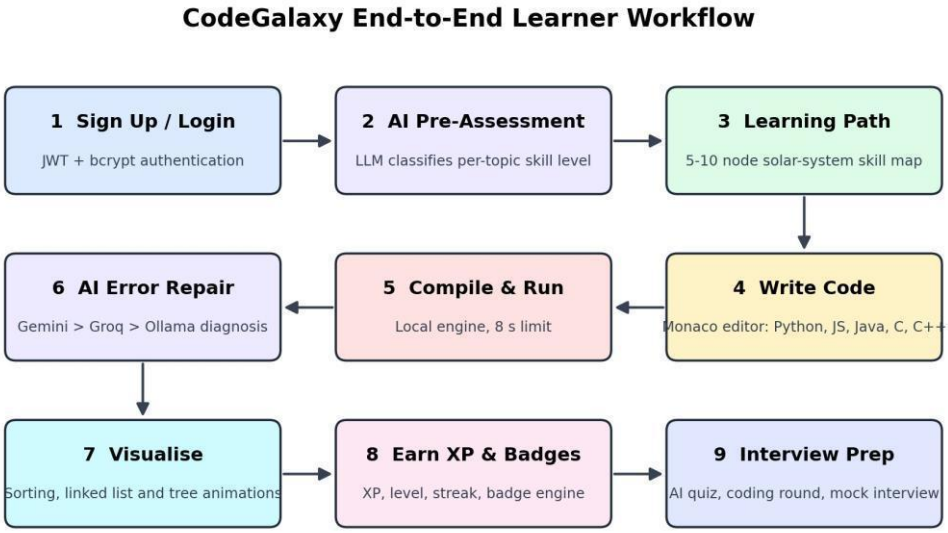


Fig. 2 End-to-End Learner Workflow

Explanation of the workflow

1. **Sign Up / Login:** The learner registers or logs in. Passwords are hashed and a JWT is issued. Outcome: an authenticated session.

2. **AI Pre-Assessment:** The learner answers a questionnaire and the LLM classifies skill level and confidence for each topic. Outcome: a skill map and preferred pace.

3. **Learning Path:** A 5 to 10 node path of prerequisite-linked skills is generated; the first node is unlocked and the rest are locked. Outcome: the solar-system map.

4. **Write Code:** The learner opens a mission, reads the brief and writes code in the Monaco editor in the chosen language.

5. **Compile & Run:** The code is sent to the execution engine, which returns stdout, stderr and time.

6. **AI Error Repair:** Only when the run fails, the source, language and error go to the AI router, which returns a structured diagnosis.

7. **Visualise:** Algorithm and data-structure visualisers animate behaviour with playback controls.

8. **Earn XP & Badges:** A passing run awards XP and updates level, streak and badges.

9. **Interview Prep:** The learner can practise adaptive quizzes, coding rounds and mock interviews.

The Agentic AI orchestration that sits behind steps 2, 6 and 9 is illustrated in Fig. 3. Each provider is tried in order, and a mock generator guarantees a valid response even when all providers fail.

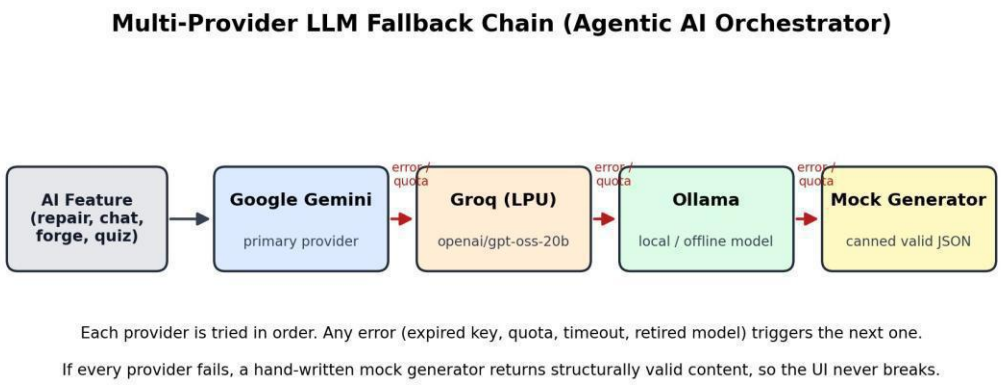


Fig. 3 Multi-Provider LLM Fallback Chain

3.3.2 Data Flow Diagram

The data flow diagram in Fig. 4 shows how data moves between the learner, the frontend, the backend engine, the AI orchestrator and the database. The seven numbered flows are explained below.

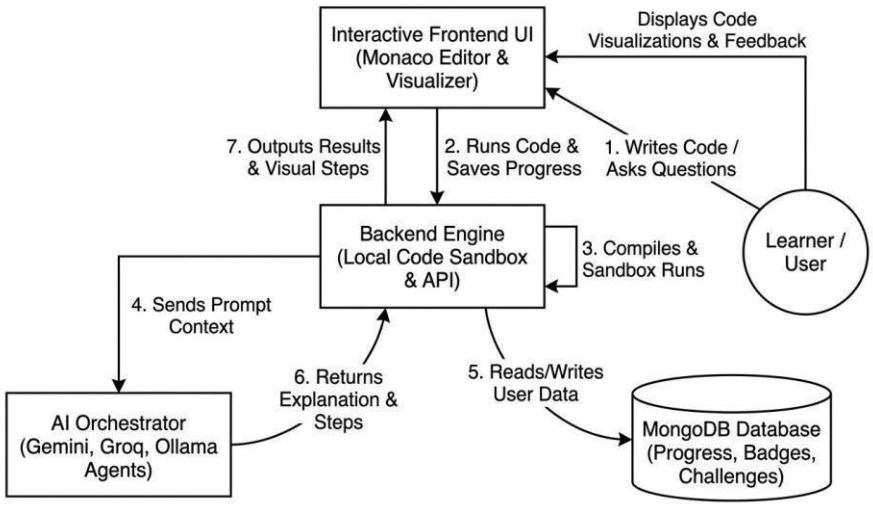


Fig. 4 Data Flow Diagram of CodeGalaxy

1. **Writes code / asks questions:** the learner enters code or a question in the interactive frontend (Monaco editor and visualiser).

2. **Runs code and saves progress:** the frontend sends the code, language and input to the backend engine and requests that progress be saved.

3. **Compiles and sandbox runs:** the backend compiles or interprets the code in the local execution engine and captures stdout, stderr and elapsed time.

4. **Sends prompt context:** on a failure, or when the learner asks for help, the backend packages code, language, error text and test-case context for the AI orchestrator.

5. **Reads and writes user data:** the backend reads and updates progress, badges and challenge data in MongoDB.

6. **Returns explanation and steps:** the AI orchestrator returns a structured diagnosis, hints or a generated challenge.

7. **Outputs results and visual steps:** the frontend displays console output, visual steps, AI feedback and updated progress.

Overall, the data flow diagram shows that execution results, AI reasoning and progress data are combined in the backend before they are returned to the learner in a single response.

3.3.3 Sequence Diagram

The sequence diagram in Fig. 5 shows the interaction between the Learner, Frontend UI, Backend API, Local Sandbox, AI Orchestrator and MongoDB database during a code run that fails and triggers AI repair.

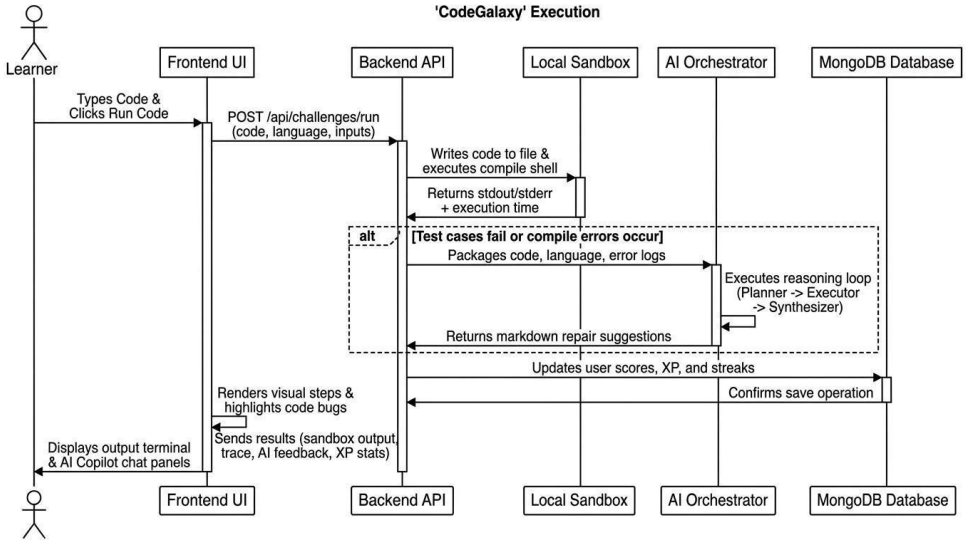


Fig. 5 Sequence Diagram of a Code Run with AI Repair

1. **Code submission:** the learner types code and clicks Run Code; the Frontend UI sends a POST request to /api/challenges/run with the code, language and inputs.

2. **Execution:** the Backend API writes the code to a temporary file and executes it in the local sandbox, which returns stdout, stderr and execution time.

3. **Conditional AI repair (alt block):** only if test cases fail or compile errors occur, the backend packages the code, language and error logs for the AI Orchestrator, which returns Markdown repair suggestions.

4. **Persistence:** the backend updates the learner's scores, XP and streaks in MongoDB, which confirms the save.

5. **Rendering:** the Frontend UI renders visual steps, highlights code issues and displays the terminal output and AI Copilot panels.

In the current implementation the repair step is a single structured-JSON prompt routed through the multi-provider chain, as described in Section 3.4. The planner-executor-synthesizer loop shown conceptually in the diagram exists in the backend as a separate module and is the planned upgrade path for this step.

3.3.4 Activity Diagram

The activity diagram in Fig. 6 describes the flow of activities during a challenge attempt, from selecting a mission to completing it.

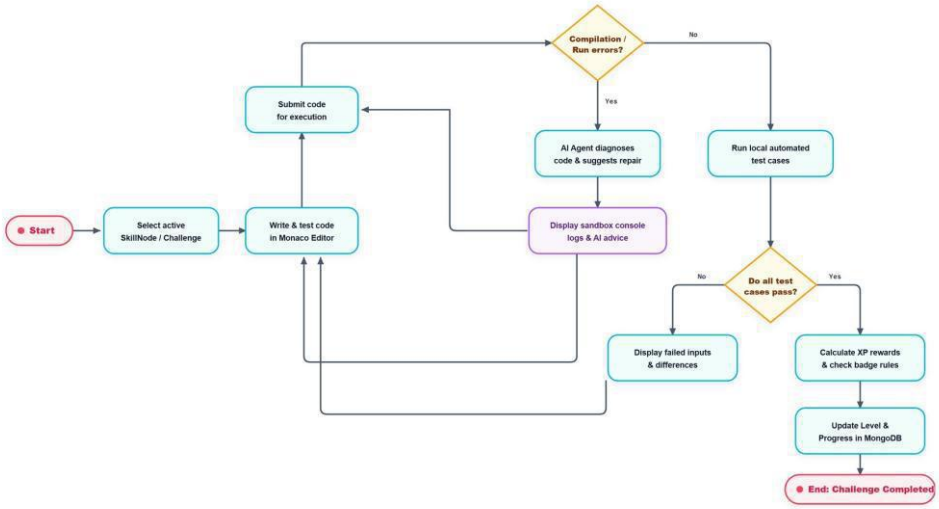


Fig. 6 Activity Diagram of a Challenge Submission 1.**Start:** the learner selects an active skill node or challenge.

2.**Write and test code:** the learner writes the solution in the Monaco editor and submits the code for execution.

3.**Compilation or run errors?** if yes, the AI agent diagnoses the code and suggests a repair; the sandbox console logs and the AI advice are displayed and the learner returns to editing.

4.**Run automated test cases:** if there are no errors, the local test cases are executed. 5.**All test cases pass?** if not, the failed inputs and differences are displayed and the learner retries; if yes, XP rewards are calculated and badge rules are checked.

6.**Update level and progress:** progress and level are stored in MongoDB and the challenge is marked complete.

3.3.5 Use Case Diagram

The use case diagram in Fig. 7 groups the platform's functionality around two human actors, the Learner/Student and the Teacher/Admin, and two supporting systems, the Local Sandbox and the AI Agent System.

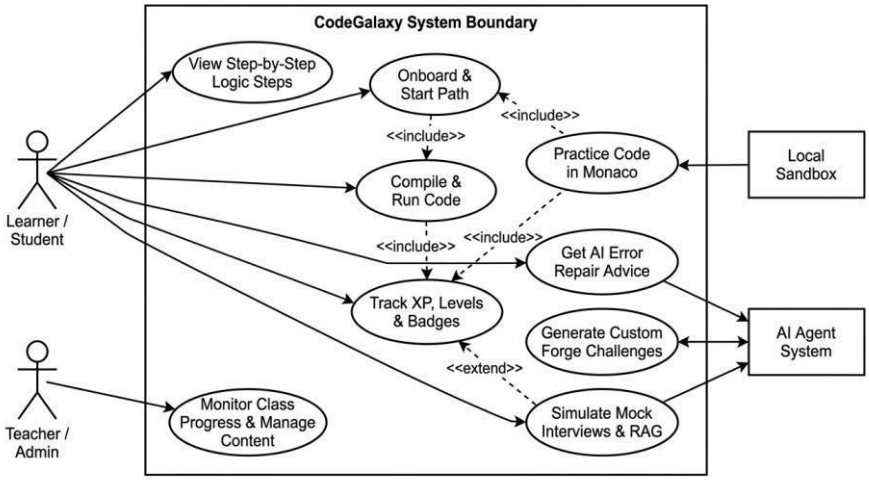


Fig. 7 Use Case Diagram of CodeGalaxy

The learner can onboard and start a learning path, practise code in Monaco, compile and run code, view step-by-step logic, track XP, levels and badges, generate custom Forge challenges and simulate mock interviews with RAG support. "Get AI Error Repair Advice" is modelled as an extension of "Compile & Run Code" because repair is only triggered conditionally, on a failing run. The teacher or administrator can monitor class progress and manage content.

3.3.6 ER Diagram

Fig. 8 shows the entity-relationship (class) diagram of the core MongoDB collections defined with Mongoose.

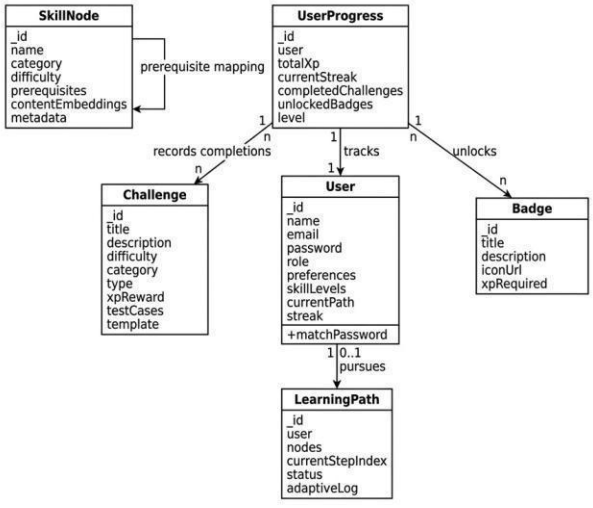


Fig. 8 Entity-Relationship Diagram of CodeGalaxy

The User document stores identity, role, learning preferences, a map of per-topic skill levels and a reference to the active LearningPath. UserProgress is a separate one-to-one collection that tracks cumulative XP, current streak, level, completed challenges and unlocked badges, which keeps the authentication document small. SkillNode models a prerequisite-linked skill graph through a self-reference. Challenge stores problem definitions, test cases, XP reward and starter template, and Badge stores achievement definitions. A contentEmbeddings field on SkillNode is reserved for the future semantic-search upgrade.

3.3.7 REST API Design

The backend exposes 40 REST endpoints across nine route modules, mounted under the /api prefix in the server entry point. Table 3.1 lists them with the access level enforced by the authentication middleware.

Table 3.1 REST endpoints of the CodeGalaxy backend.

Module (base path)	Method	Endpoint	Access
Auth (/api/auth)	POST	/signup	Public
Auth (/api/auth)	POST	/login	Public
Auth (/api/auth)	POST	/social	Public
Auth (/api/auth)	GET	/me	JWT
AI (/api/ai)	POST	/chat	Public
AI (/api/ai)	POST	/assess/pre	JWT
AI (/api/ai)	POST	/path/generate	JWT
AI (/api/ai)	GET	/path/current	JWT
AI (/api/ai)	POST	/quiz/submit	JWT
AI (/api/ai)	POST	/forge/generate	Optional JWT
AI (/api/ai)	POST	/copilot/analyze	Optional JWT
AI (/api/ai)	POST	/repair/diagnose	Optional JWT
Challenges (/api/challenges)	GET	/	Public
Challenges (/api/challenges)	POST	/	Admin
Challenges (/api/challenges)	GET	/:id	Public
Challenges (/api/challenges)	POST	/run	JWT
Progress (/api/progress)	GET	/	JWT
Progress (/api/progress)	POST	/	JWT
Learning (/api/learning)	GET	/challenges	Public
Learning (/api/learning)	GET	/levels	Public
Learning (/api/learning)	GET	/categories	Public
Learning (/api/learning)	GET	/levels/:id	Public

Module (base path)	Method	Endpoint	Access
Learning (/api/learning)	GET	/challenges/:id	Public
Learning (/api/learning)	GET	/challenges/:id/next	Public
Learning (/api/learning)	GET	/progress	Optional JWT
Learning (/api/learning)	POST	/submit	JWT
Gamification (/api/gamification)	GET	/progress	JWT
Gamification (/api/gamification)	GET	/badges	JWT
Gamification (/api/gamification)	GET	/leaderboard	JWT
Interview (/api/interview-prep)	GET	/state	JWT
Interview (/api/interview-prep)	POST	/config	JWT
Interview (/api/interview-prep)	POST	/quiz/generate	JWT
Interview (/api/interview-prep)	POST	/quiz/submit	JWT
Interview (/api/interview-prep)	POST	/coding/generate	JWT
Interview (/api/interview-prep)	POST	/coding/evaluate	JWT
Interview (/api/interview-prep)	POST	/interview/answer	JWT
RAG (/api/rag)	POST	/upload	JWT
RAG (/api/rag)	POST	/ask	JWT
RAG (/api/rag)	DELETE	/:docId	JWT
Analytics (/api/analytics)	GET	/teacher	Teacher or Admin

3.3.8 Database Design (Data Dictionary)

Table 3.2 gives the data dictionary of the MongoDB collections used by the platform. User identity and learning progress are deliberately kept in separate collections so that the authentication document stays small.

Table 3.2 Data dictionary of CodeGalaxy collections.

Collection	Key fields	Purpose
User	name, email, password (hashed), role / isAdmin, preferences, skillLevels (Map), currentPath, streak	Identity, preferences and skill map
UserProgress	user (unique), totalXp, currentStreak, level, completedChallenges[], unlockedBadges[]	Gamification state
LearningPath	user, nodes[], currentStepIndex, status, adaptiveLog	Generated skill sequence per learner
SkillNode	name, category, difficulty, description, prerequisites[], contentEmbeddings[], metadata	Prerequisite-linked skill graph
Challenge	title, description, difficulty, category, type, xpReward, instructions, testCases, template	Curated and generated problems
Badge	title, description, iconUrl, xpRequired	Achievement definitions

Collection	Key fields	Purpose
InterviewPrep	per-user configuration, quiz results and interview state	AI interview-prep state

3.4 Agentic AI Layer Design

The Agentic AI layer is what distinguishes this version of CodeGalaxy from a plain visual-learning tool. It is designed around four cooperating agent roles that share one orchestration router. Table 3.3 summarises each role, its design intent and the way it is realised in the current source.

Table 3.3 Agent roles: design intent versus current realisation.

Agent role	Design intent	Realisation in the current system
Code Analysis Agent	Detect syntax and logic errors and suggest fixes	Structured-JSON analysis and debug prompts (code feedback and repair diagnosis) through the shared router
Execution Visualization Agent	Show variables, loops and data structures in real time	Client-side React visualisers for sorting, linked lists and trees; no server-side AI needed
Learning Assistant Agent	Answer questions and give personalised guidance	Chat replies (with streaming) and TF-IDF grounded document Q&A, plus vision-model image Q&A
Assessment Agent	Evaluate skills and generate challenges	Skill profiling, adaptive quizzes, Nebula Forge challenge generation, coding evaluation and interview simulation

Behind these roles are three orchestration mechanisms. First, the multi-provider router gives every agent role resilience through the Gemini, Groq, Ollama and mock fallback chain. Second, the planner-executor-synthesizer loop implements genuine multi-step reasoning: a plan is produced, each step is executed with the results of the previous steps as context, and the outputs are synthesised into one deliverable. Third, a three-step prompt chain (summary, key points, quiz questions) demonstrates output-to-input chaining for learning-material processing.

At present the four agent roles are served by single structured prompts through the shared router, which keeps latency low and behaviour predictable for learners. The planner-executor-synthesizer loop and the prompt chain are implemented in the backend and are not yet reachable from a live route. Connecting the repair and copilot endpoints to the agent loop is the first item of future work and is described in Chapter 6.

Table 3.4 lists the AI functions exported by the shared router module and the feature each one serves. Every JSON-producing function has a hand-written mock fallback, so each feature stays usable even when all providers fail.

Table 3.4 AI functions served by the shared router.

Function	Purpose	Used by
generateSkillProfile	Classify per-topic skill level and confidence	Pre-assessment
generateLearningPath	Create a 5 to 10 node skill sequence	Learning path
getRemedialSuggestion	Simplified explanation after a failed quiz	Adaptive quiz
generateChatReply / ...Stream	Conversational assistant, optionally streamed	AI chatbot
generateCustomChallenge	Challenge with description, template and tests	Nebula Forge
analyzeCodeFeedback	Hints, logic feedback and complexity	Copilot panel
debugCodeError	Step-by-step repair diagnosis	Repair agent
generateInterviewQuiz	Adaptive multiple-choice quiz	Interview prep
generateInterviewCoding	Coding problem at a chosen difficulty	Interview prep
evaluateInterviewCode	Logic, style and complexity scoring	Interview prep
generateInterviewQuestion / evaluateSimulatedInterview	Mock interview turns and final evaluation	Interview prep
generateWithSystemPrompt	Generic single-turn call	RAG, agent loop, prompt chain
checkAiProviders	Start-up health check of each provider	Server start-up

3.5 Security & Reliability Design

Because CodeGalaxy executes user-submitted code and calls external AI services, security and reliability were designed in from the start. The main controls are:

- **Authentication and password storage.** Passwords are hashed with bcrypt (10 salt rounds) in a Mongoose pre-save hook. Private routes verify a signed JWT; an optional-authentication variant lets guests use selected AI features; admin and teacher routes add role checks.
- **Execution controls.** Each submission uses a uniquely named temporary file, an 8-second timeout and a 10 MB output cap, and temporary files and compiled binaries are deleted shortly after the run.
- **Secret management.** API keys, the JWT secret and the database URI are read from environment variables, with an example file provided instead of real credentials.

- **Input validation.** Controllers reject requests with missing fields using HTTP 400, upload size is capped, and an async error handler returns consistent error responses.
- **Reliability.** The provider fallback chain, mock generators and the start-up diagnostic keep every AI feature available and make misconfiguration visible.

The current limitations are equally important. Code runs as a child process of the server and is not container-isolated, there is no per-user rate limiting, uploaded documents live in server memory, and no automated test suite exists yet. These are listed as the highest-priority hardening items in Chapter 6.

CHAPTER - 4
APPLICATION CODE

4.1 Coding Blocks (Frontend & Backend Code)

This section presents the key code blocks of CodeGalaxy, taken from the project repository. Long files are shortened to the parts that matter for understanding the design. Table 4.1 maps the main backend files to their role and size.

Table 4.1 Main backend files, their role and size.

File	Role	Lines
utils/executeCode.js	Multi-language code execution engine	205
services/geminiService.js	Provider-agnostic AI router, all AI functions and mock fallbacks	1,002
services/groqService.js	Groq chat, JSON and vision calls	237
services/ollamaService.js	Local Ollama calls (offline fallback)	219
services/agentService.js	Planner-executor-synthesizer agent loop	90
services/promptChainService.js	Summary, key points and quiz prompt chain	37
services/ragService.js	Chunking, TF-IDF index, retrieval and image Q&A	300
services/qdrantService.js	Scaffolded vector store client	67
controllers/aiController.js	Assessment, path, forge, copilot and repair handlers	304

4.1.1 Code Execution Engine (backend/utils/executeCode.js)

The engine runs each submission through the correct toolchain with an 8-second timeout and a 10 MB output cap. Only a non-zero exit, a timeout kill or a spawn failure is treated as failure, because stderr output alone can be a harmless compiler warning.

```
const runShell = (command, options = {}) => {
 return new Promise((resolve, reject) => {
    exec(command, { timeout: 8000, maxBuffer: 1024 * 1024 * 10, ...options },
    (error, stdout, stderr) => {
    if (error) {
    if (error.killed || error.signal === 'SIGTERM') {
    reject({ error, stderr: stderr ||
          'Execution timed out (exceeded 8s time limit).' });
    return;
    }
    reject({ error, stderr: stderr || error.message });
    return;
    }
    // stderr alone is NOT a failure: compilers print warnings there
resolve(stderr ? `${stdout}${stdout && stderr ? '\n' : ''}${stderr}`
    : stdout);
    });
 });
};
```

Language dispatch inside runCode()

```
const startTime = Date.now();
try {
 if (format === 'cpp')       result = await executeCpp(filepath, inputFilePath);
else if (format === 'c')    result = await executeC(filepath, inputFilePath); else
if (format === 'py')   result = await executePython(filepath, inputFilePath); else
if (format === 'js')
    result = await executeJavaScript(filepath, inputFilePath);
 else if (format === 'java') result = await executeJava(filepath, inputFilePath);
else throw new Error(`Unsupported language: ${language}`);
} catch (err) {
 isError = true;
 result = err.stderr || err.error?.message || err.message;
}
const timeTaken = (Date.now() - startTime) / 1000;
return { output: result, success: !isError, time: timeTaken };
```

4.1.2 Multi-Provider LLM Router (backend/services/geminiService.js)

Every JSON-producing AI feature goes through callJsonModel. The AI_PROVIDER environment variable can pin a provider, while the default auto mode tries Gemini, then Groq, then Ollama.

```
const AI_PROVIDER = (process.env.AI_PROVIDER || 'auto').toLowerCase();

const callJsonModel = async (prompt) => {
 if (AI_PROVIDER === 'ollama') return ollamaService.generateJSON(prompt); if
(AI_PROVIDER === 'groq')   return groqService.generateJSON(prompt); if
(AI_PROVIDER === 'gemini') {
    const result = await jsonModel.generateContent(prompt);
    return (await result.response).text();
 }
 // auto: Gemini first, then Groq, then Ollama as silent fallbacks
 try {
    const result = await jsonModel.generateContent(prompt);
    return (await result.response).text();
 } catch (geminiError) {
    console.warn("Gemini failed, trying Groq:", geminiError?.message);
try {
    return await groqService.generateJSON(prompt);
    } catch (groqError) {
    console.warn("Groq failed, falling back to Ollama:", groqError?.message);
    return ollamaService.generateJSON(prompt);
    }
 }
};
```

4.1.3 AI Repair Diagnosis (aiController.js and geminiService.js)

The repair endpoint validates the request and calls debugCodeError, whose prompt asks for a diagnosis and ordered steps but explicitly not the solved code. If every provider fails, a hand-written mock diagnosis is returned.

backend/controllers/aiController.js

```
const repairActiveCode = asyncHandler(async (req, res) => {
   const { code, language, errorMsg, testCases } = req.body;
    if (!code || !language) {
      res.status(400);
      throw new Error('Code and language are required');
    }
    const diagnosis = await geminiService.debugCodeError(
      code, language, errorMsg || 'Unknown execution error', testCases || []
    );
    if (!diagnosis) {
      res.status(500);
```

```
    throw new Error('AI debugging failed');
  }
  res.json(diagnosis);
});
```

backend/services/geminiService.js (end of debugCodeError)

```
  // Prompt requires strict JSON: { diagnosis, steps[], remedialCodeSnippet }
  // "Provide direct, step-by-step diagnostic actions (without giving the
  //  full solved code)."
  try {
    const text = await callJsonModel(prompt);
    return parseAiJson(text) || getMockDebugError(code, language, errorMsg);
  } catch (error) {
    console.error("Gemini Debug Error (Using local fallback):", error);
    return getMockDebugError(code, language, errorMsg);
  }
```

4.1.4 Planner-Executor-Synthesizer Agent (backend/services/agentService.js)

The agent loop plans a task, executes each step with the results of earlier steps as context and finally synthesises one deliverable. The module is fully implemented and exported; connecting it to a route is planned work.

```
const runAgent = async (task) => {
 // STEP 1: Plan - ask the LLM for 3 to 6 ordered steps (JSON array) const
planRaw = await geminiService.generateWithSystemPrompt(
    PLAN_SYSTEM_PROMPT, `Task: ${task}`);
 const plan = parsePlan(planRaw);
 if (plan.length === 0) throw new Error('Agent failed to produce a plan.');


  // STEP 2: Execute each step, feeding earlier results into later steps
 const stepResults = [];
  for (let i = 0; i < plan.length; i++) {
    const priorContext = stepResults
    .map((r, idx) => `Step ${idx + 1} ("${plan[idx]}") result:\n${r}`)
    .join('\n\n');
    const userPrompt = [
    `Overall Task: ${task}`,
    `Full Plan:\n${plan.map((s, k) => `${k + 1}. ${s}`).join('\n')}`,
    priorContext ? `\nResults so far:\n${priorContext}` : '',
    `\nNow execute step ${i + 1}: "${plan[i]}"`,
    ].filter(Boolean).join('\n');
    const result = await geminiService.generateWithSystemPrompt(
    EXECUTE_SYSTEM_PROMPT, userPrompt);
    stepResults.push(result.trim());
  }


  // STEP 3: Finalize - synthesize all step outputs into one deliverable
 const finalOutput = await geminiService.generateWithSystemPrompt(
 FINALIZE_SYSTEM_PROMPT, finalPrompt);
  return { task, plan, stepResults, finalOutput: finalOutput.trim() }; };
```

4.1.5 TF-IDF Retrieval for Document Q&A (backend/services/ragService.js)

The retrieval pipeline needs no vector database. Chunks are weighted with TF-IDF, a query is vectorised the same way and chunks are ranked by cosine similarity.

```
const tokenize = (text) =>
  text.toLowerCase().replace(/[^a-z0-9\s]/g, ' ')
      .split(/\s+/).filter((w) => w.length > 2);

// idf(term) = ln((N + 1) / (df + 1)) + 1
docFreq.forEach((df, term) => idf.set(term, Math.log((N + 1) / (df + 1)) + 1));
```

```
const cosineSimilarity = (a, b) => {
  let dot = 0;
  const [smaller, larger] = a.vec.size < b.vec.size ? [a, b] : [b, a];
 smaller.vec.forEach((weight, term) => {
    if (larger.vec.has(term)) dot += weight * larger.vec.get(term); });
 return dot / (a.norm * b.norm);
};

const retrieveRelevantChunks = (doc, query, topK = 4) => {
 const queryVec = vectorizeQuery(query, doc.idf);
 const scored = doc.chunks.map((chunk, i) => ({
    chunk, score: cosineSimilarity(queryVec, doc.chunkVectors[i]),
 }));
 scored.sort((a, b) => b.score - a.score);
 // if nothing overlaps, fall back to the first chunks so the model has context
const meaningful = scored.filter((s) => s.score > 0);
 return (meaningful.length ? meaningful : scored)
    .slice(0, topK).map((s) => s.chunk);
};
```

4.1.6 Gamification Data Model (backend/models/UserProgress.js)

```
const userProgressSchema = new mongoose.Schema({
 user:          { type: mongoose.Schema.Types.ObjectId, ref: 'User',
    required: true, unique: true },
 totalXp:       { type: Number, default: 0 },
 currentStreak: { type: Number, default: 0 },
 level:         { type: Number, default: 1 },
 completedChallenges: [{
    challenge:   { type: mongoose.Schema.Types.ObjectId,
    ref: 'LearningChallenge' },
    completedAt: { type: Date, default: Date.now },
    score: Number
 }],
 unlockedBadges: [{
    badge:       { type: mongoose.Schema.Types.ObjectId, ref: 'Badge' },
    unlockedAt:  { type: Date, default: Date.now }
 }]
}, { timestamps: true });
```

4.1.7 Authentication (User.js and authMiddleware.js)

Passwords are hashed with bcrypt before they are stored, and the protect middleware verifies the bearer token on every private route and attaches the user, without the password hash, to the request.

backend/models/User.js

```
userSchema.pre('save', async function () {
  if (!this.isModified('password')) return;
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});
```

backend/middleware/authMiddleware.js

```
const protect = async (req, res, next) => {
 if (req.headers.authorization &&
      req.headers.authorization.startsWith('Bearer')) {
    try {
      const token = req.headers.authorization.split(' ')[1];
      if (!token || token === 'null' || token === 'undefined') {
      return res.status(401).json({ message: 'Not authorized, no token' });
      }
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
     req.user = await User.findById(decoded.id).select('-password');
     if (!req.user) {
```

```
        return res.status(401).json({ message: 'Not authorized, user not found' });
      }
      return next();
    } catch (error) {
      return res.status(401).json({ message: 'Not authorized, token failed' });
    }
  }
 return res.status(401).json({ message: 'Not authorized, no token' }); };
```

4.1.8 AI Routes and Server Bootstrap (aiRoutes.js, server.js)

```
router.post('/chat', aiController.chatWithAI);

// AI Learning Adaptation Routes
router.post('/assess/pre',    protect, aiController.submitPreAssessment);
router.post('/path/generate', protect, aiController.generatePath);
router.get('/path/current',   protect, aiController.getCurrentPath);
router.post('/quiz/submit',   protect, aiController.submitQuiz);

// Agentic AI routes (challenge generation, co-pilot, debugging diagnostics)
router.post('/forge/generate',  optionalProtect, aiController.createForgeChallenge);
router.post('/copilot/analyze', optionalProtect, aiController.analyzeActiveCode);
router.post('/repair/diagnose', optionalProtect, aiController.repairActiveCode);
```

backend/server.js

```
app.use('/api/auth', authRoutes);
app.use('/api/ai', require('./routes/aiRoutes'));
app.use('/api/challenges', require('./routes/challengeRoutes'));
app.use('/api/progress', require('./routes/progressRoutes'));
app.use('/api/learning', require('./routes/learningRoutes'));
app.use('/api/gamification', require('./routes/gamificationRoutes'));
app.use('/api/analytics', require('./routes/analyticsRoutes'));
app.use('/api/interview-prep', require('./routes/interviewPrepRoutes'));
app.use('/api/rag', require('./routes/ragRoutes'));

app.listen(PORT, () => {
 console.log(`Server running on port ${PORT}`);
 // Fire-and-forget: logs provider health to the console at start-up
require('./services/geminiService').checkAiProviders()
    .catch(e => console.error('[AI] Diagnostic check crashed:', e.message));
});
```

4.1.9 Nebula Forge Challenge Generator (aiController.js)

Nebula Forge turns a plain-English prompt into a complete challenge object, which is saved to MongoDB and returned to the client so that it can be played immediately.

```
const createForgeChallenge = asyncHandler(async (req, res) => {
const { prompt } = req.body;
  if (!prompt) { res.status(400); throw new Error('Prompt is required'); }

  const generated = await geminiService.generateCustomChallenge(prompt);
 if (!generated) {
    res.status(500);
    throw new Error('Could not generate challenge using the AI agent'); }
 const challenge = new Challenge({
    title: generated.title,
    description: generated.description,
    difficulty: generated.difficulty || 'Easy',
    category: 'Nebula Forge',
    type: 'Algorithm',
    xpReward: generated.xpReward || 100,
    instructions: generated.instructions || '',
    testCases: generated.testCases || [],
    template: generated.template || ''
  });
```

```
  const savedChallenge = await challenge.save();
  res.status(201).json(savedChallenge);
});
```

4.1.10 Algorithm Visualizer Step Generation (AlgorithmVisualizer.tsx)

The visualiser pre-computes a list of steps and then plays them back. Each step records the array, the indices being compared or swapped, a description and the pseudocode line to highlight. The code below shows the bubble sort generator.

```
function generateSortingSteps(algo: SortingAlgo, arr: number[]): Step[] {
const s: Step[] = [];
 let copy = [...arr];
 if (algo === 'bubble') {
    s.push({ array: [...copy], description: 'Initial array state',
    highlightLine: 0 });
    for (let i = 0; i < copy.length; i++) {
    for (let j = 0; j < copy.length - i - 1; j++) {
    s.push({ array: [...copy], comparing: [j, j + 1],
        description: `Compare ${copy[j]} & ${copy[j + 1]}`,
    highlightLine: 2 });
    if (copy[j] > copy[j + 1]) {
    [copy[j], copy[j + 1]] = [copy[j + 1], copy[j]];
    s.push({ array: [...copy], swapping: [j, j + 1],
             description: `Swapped ${copy[j + 1]} & ${copy[j]}`,
        highlightLine: 3 });
    }
    }
    }
    s.push({ array: [...copy],
        sorted: Array.from({ length: copy.length }, (_, i) => i),
    description: 'Array sorted!', highlightLine: 0 });
 }
 // other algorithms use the same Step structure and player
 return s;
}
```

4.1.11 Frontend Repair Agent Call (frontend/src/components/editor/Terminal.tsx)

When a run fails, the learner can deploy the repair agent from the console. The frontend sends the code, language, error text and test cases and renders the structured diagnosis; if the call fails, a safe default message is shown.

```
const handleDeployRepairAgent = async () => {
 if (!code) return;
 setShowAgent(true);
 setIsRepairing(true);
 try {
    const errorMsg = result?.compile_output || result?.stderr
    || result?.message || "Unknown execution error";
const res = await api.post('/ai/repair/diagnose', {
    code, language, errorMsg, testCases
    });
    setDiagnosisData(res.data);
 } catch (err) {
    console.error("Repair agent failure", err);
    setDiagnosisData({
    diagnosis: "Diagnosis link lost. Logic core reported general failure.",
    steps: ["Check compiler log files manually.",
    "Ensure variables are initialized and imported properly."] }); } finally
{
    setIsRepairing(false);
 }
};
```

4.2 Execution Flow

Initial State Setup: The React application loads, restores the learner's JWT session and fetches progress, learning path and challenge data from the REST API. State hooks hold the current code, language, execution result and diagnosis.

Code Input & Language Selection: When the learner types in the Monaco editor or changes the language, the component state is updated with the latest code and language.

Code Execution Trigger: When the learner clicks Run, the frontend posts the code, language and input to the backend. The Express router applies the authentication middleware and hands the request to the execution engine.

Execution and Result: The engine writes a temporary file, runs the correct toolchain and returns output, success flag and elapsed time. Temporary source files, input files and compiled binaries are deleted shortly afterwards.

AI Diagnosis on Failure: If the run failed, the frontend posts the code, language and error to /api/ai/repair/diagnose. The router tries Gemini, then Groq, then Ollama, and finally the mock generator, and returns a diagnosis, ordered steps and a conceptual snippet.

Visualisation and Output Display: The output, execution status and visual state are displayed in the workspace. Algorithm visualisers play recorded states with playback controls.

Progress Tracking: On a passing submission the learner receives XP, and level, streak and badges are updated and stored in the progress collection.

Rendering the Interface: The editor, console, visualiser, AI assistant and dashboard components re-render automatically as state changes.

4.3 Configuration, Build and Deployment

CodeGalaxy is configured through environment variables, so no secret is stored in the source code. Table 4.2 lists the variables defined in the example files of the repository.

Table 4.2 Environment variables of CodeGalaxy.

Variable	Meaning
PORT	Port on which the Express server listens (default 5000)
MONGO_URI	MongoDB Atlas connection string
JWT_SECRET	Secret used to sign and verify JSON Web Tokens
NODE_ENV	Runtime mode (development or production)

Variable	Meaning
AI_PROVIDER	auto (default), gemini, groq or ollama; selects or pins the AI provider
GEMINI_KEY	Google Gemini API key (primary provider)
GROQ_API_KEY	Groq API key (fallback provider and image Q&A)
VITE_API_URL	Frontend setting: base URL of the backend API

The system is set up and run with the following steps:

1. Install Node.js 18 or higher together with the toolchains for the languages to be executed (Python 3, JDK, gcc and g++).

2. In the backend folder run npm install, copy .env.example to .env and fill in the values from Table 4.2.

3. Start the backend with npm run dev (nodemon) during development or npm start in production; the start-up diagnostic reports which AI providers are reachable.

4. In the frontend folder run npm install, set VITE_API_URL and start the development server with npm run dev.

5. Create the production build with npm run build. The frontend is deployed on Vercel with a rewrite of all routes to index.html, and the backend is deployed on Render.

6. Optionally install Ollama and pull a local model so that the platform keeps working offline when no cloud provider is reachable.

CHAPTER - 5
TESTING AND RESULTS

5.1 Resulting Screens

5.1.1 CodeGalaxy Home Page

The home page introduces the platform with the headline “Master Algorithms in Orbit” and gives access to the learning paths, challenges and archives.



Fig. 9 CodeGalaxy Home Page

5.1.2 Mission Control Dashboard

The dashboard shows total XP, current level, daily streak and completed challenges, a multi-vector performance chart and a knowledge-core donut that summarises per-topic progress.

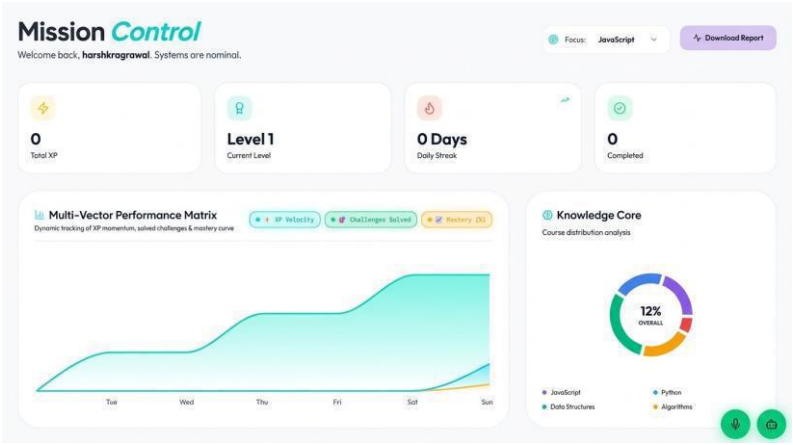


Fig. 10 Mission Control Dashboard

5.1.3 Challenge Workspace

The challenge workspace combines the problem brief, a real-time visualiser, the Monaco editor with language selection and a console that shows execution status, time and memory.

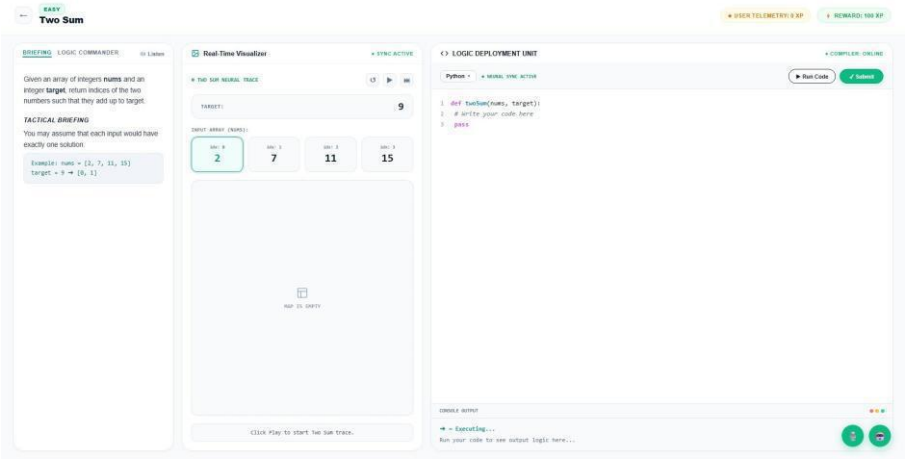


Fig. 11 Challenge Workspace with Editor, Visualizer and Console

5.1.4 Algorithm Visualizer

The DSA visualiser animates sorting and searching with colour-coded active, comparing and sorted states, a step counter, playback controls, custom datasets and time and space complexity information. Bubble sort shows every comparison and swap; the other algorithms use the same player with progressive steps.



Fig. 12 Interactive DSA Algorithm Visualizer

5.1.5 AI Conversational Assistant

The AI assistant accepts text and voice input and answers in formatted Markdown, for example with a side-by-side comparison table of two programming languages, so that the learner can resolve questions without leaving the workflow.

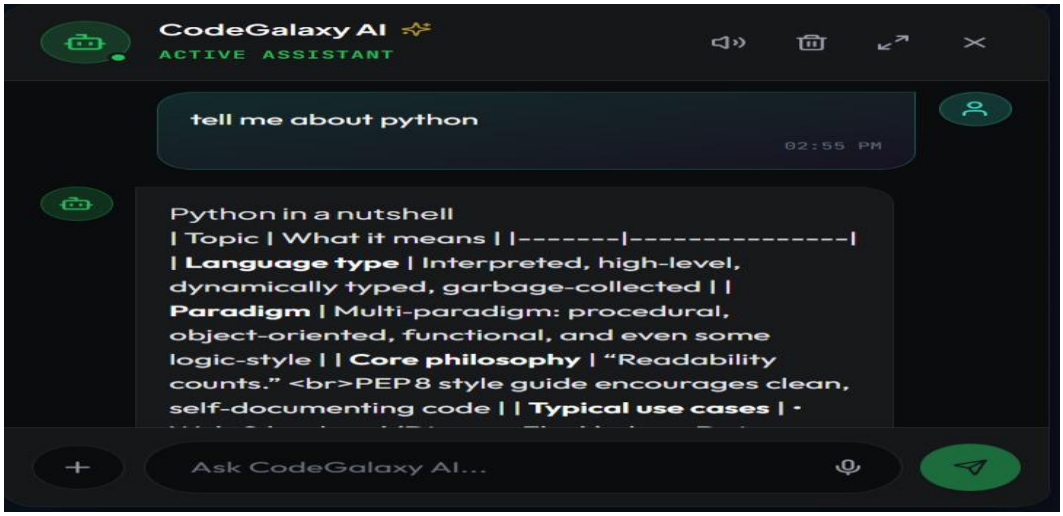


Fig. 13 AI Conversational Assistant

5.2 Result Analysis

The results of CodeGalaxy are reported from the working prototype and from a source-level review of the repository. A formal classroom study with learners has not yet been conducted and is planned as future work, so the observations below describe verified system behaviour rather than measured learning gains.

1. Real Execution with Honest Failure Detection. The execution engine compiles and runs five languages with an 8-second limit and treats only real failures as errors, which avoids false error reports from compiler warnings.

2. Guided AI Diagnosis. Failing runs are turned into structured diagnoses with ordered steps and a conceptual snippet, without exposing the solved program.

3. Resilient AI Layer. The Gemini, Groq, Ollama and mock fallback chain absorbed two live model-deprecation incidents. Because these initially masked a failure, a start-up diagnostic was added that tests each provider and reports its status in the server console.

4. Grounded Document and Image Q&A. The TF-IDF retrieval pipeline provides genuine lexical retrieval without a vector database, and an instructed system prompt makes the model state when the context is insufficient.

5. Engaging Progress Model. XP, levels, streaks, badges, the leaderboard and the solar-system map are stored in dedicated collections and shown on the dashboard.

6. Career Readiness. The interview simulator provides seven endpoints for configuration, adaptive quizzes, generated coding problems, evaluation and a turn-by-turn interview, each with a mock fallback.

5.2.1 Result Summary

Table 5.1 summarises the size of the implemented system, measured as JavaScript, TypeScript and JSX lines in the repository excluding dependencies, build output and temporary folders.

Table 5.1 Implementation size of CodeGalaxy.

Measure	Value	Share / note
Total lines of code	28,769	Frontend 78%, Backend 22%
Frontend lines	22,459	Components 11,208; Pages 9,138; Hooks 550; Shell, utils, types 1,563
Backend lines	6,310	Services 1,942; Controllers 1,425; Models 407; Server and scripts 2,144
REST endpoints	40	Across 9 backend modules
Frontend pages	30	Landing, dashboard, workspace, interview pages, teacher view
Supported languages	5	Python, JavaScript, Java, C, C++
LLM providers in chain	3	Gemini, Groq, Ollama (plus mock fallback)

Table 5.2 Implementation status of CodeGalaxy modules.

Module	Status	Note
Authentication (JWT + bcrypt)	Live	Used by all protected routes
Code execution engine	Live	Host child process, 8 s limit
Multi-provider LLM router	Live	Gemini > Groq > Ollama > mock
TF-IDF document RAG	Live	In-memory, single instance
Gamification engine	Live	XP, streak, badges, leaderboard
Interview simulator	Live	Seven endpoints, voice in/out
Agent loop (planner-executor)	Implemented, not wired	No route calls it yet
Prompt-chain module	Implemented, not wired	No route calls it yet
Qdrant semantic search	Implemented, not wired	Scaffolded only
Docker / container sandbox	Planned	Not yet built

In summary, six of the ten modules are live in the running application, three are implemented but not yet connected to a route, and one is planned. The platform reduces the fear and confusion often associated with programming by making failures explainable, execution visible and progress rewarding.

CHAPTER - 6
CONCLUSION & FUTURE SCOPE

6.1 Conclusion

Programming is an important skill, yet many beginners struggle because traditional teaching and output-only coding platforms do not explain how programs behave internally.

CodeGalaxy was developed to address this problem as an interactive, gamified and Agentic AI based learning platform.

The platform provides a working full-stack system with a real five-language execution engine, a resilient three-provider LLM fallback chain, guided AI error diagnosis that explains instead of spoiling, hand-built algorithm and data-structure visualisers, a lexical document and image question-answering pipeline, a persistent gamification model with a solar-system skill map, a dynamic challenge generator and an AI technical-interview simulator.

Measured against its own design documents, the platform is best described as AI-assisted today, with a clear path to fuller autonomy. The four agent roles are served by structured prompts through one shared router, and a genuine planner-executor-synthesizer loop exists in the code base but is not yet reachable from a live route. We have stated this plainly because an accurate account of what a system does today is more useful than a description of what it is designed to do eventually.

Overall, CodeGalaxy offers a practical alternative to traditional programming education by combining real execution, visual explanation, Agentic AI guidance, gamification and career preparation in one environment for students, teachers and self-learners.

6.2 Future Scope

CodeGalaxy has a strong future scope. The most important next steps are listed below, in approximate order of priority.

- **Wire the agent loop.** Connect the planner-executor-synthesizer loop to the repair and copilot endpoints so that the Agentic AI framing is backed by multi-step reasoning in the live application.
- **Isolated execution.** Replace or supplement the host child-process engine with container or gVisor isolation before accepting code from untrusted public users.
- **Semantic retrieval.** Complete the Qdrant integration and populate the embeddings field so that document Q&A and skill search use semantic rather than purely lexical matching.

- **Streaming delivery.** Move long AI responses to Server-Sent Events or WebSockets for faster perceived response.
- **Rate limiting and durability.** Add per-user rate limiting, persist uploaded documents outside process memory and add a fallback for image question answering.
- **Automated testing.** Add automated unit and integration tests for the frontend and backend.
- **Scalable visualisation.** Record per-comparison steps for selection, insertion, quick and merge sort, and generate steps lazily so that very large inputs do not exhaust browser memory.
- **Offline-first and mobile.** Add an offline-first synchronisation layer and a mobile application so that learners can practise anywhere.
- **Learning impact study.** Run a controlled study with students to measure learning gains, engagement and retention.
- **Community features.** Add collaborative coding, online competitions, discussion forums and more languages.

REFERENCES

[1] J. Jenkins, “On the Difficulty of Learning to Program,” Proceedings of the 3rd Annual Conference of the LTSN Centre for Information and Computer Sciences, 2002.

[2] T. Rajala, M.-J. Laakso, E. Kaila and T. Salakoski, “Effectiveness of Program Visualization: A Case Study,” Journal of Educational Technology & Society, 2008.

[3] P. J. Guo, “Online Python Tutor: Embeddable Web-Based Program Visualization,” ACM SIGCSE, 2013.

[4] S. Halim, F. Halim and B. C. Lau, “VisuAlgo: Visualizing Algorithms for Learning,” 2012.

[5] M. Resnick et al., “Scratch: Programming for All,” Communications of the ACM, 2009.

[6] S. Deterding, D. Dixon, R. Khaled and L. Nacke, “From Game Design Elements to Gamefulness: Defining Gamification,” MindTrek Conference, 2011.

[7] CodeCombat Inc., “CodeCombat: A Game-Based Programming Learning Platform,” 2013.

[8] S. Sentance, E. Barendsen and C. Schulte, Computer Science Education: Perspectives on Teaching and Learning, Springer, 2018.

[9] J. Sorva, “Visual Program Simulation in Introductory Programming Education,” Aalto University, 2012.

[10] A. Robins, J. Rountree and N. Rountree, “Learning and Teaching Programming: A Review and Discussion,” Computer Science Education, 2003.

[11] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems (NeurIPS), 2020.

[12] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” International Conference on Learning Representations (ICLR), 2023.

[13] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” Information Processing & Management, vol. 24, no. 5, 1988.

[14] Google, “Gemini API documentation,” https://ai.google.dev/gemini-api/docs (accessed October 2026).

[15] Groq, “Groq API documentation,” https://console.groq.com/docs (accessed October 2026).

[16] Ollama, “Ollama: run large language models locally,” https://github.com/ollama/ollama (accessed October 2026).

Github Link: - https://github.com/HarshKumar5822/CodeGalaxy-An-Agentic-AI-Based-Interactive- Programming-Learning-Platform

Live Demo Link: - https://code-galaxy-an-agentic-ai-based-int.vercel.app/

CODEGALAXY: AN AGENTIC AI BASED INTERACTIVE PROGRAMMING LEARNING PLATFORM

G. Divya Jyothi
Assistant Professor
Department of Computer Science & Engineering - Cyber Security
Malla Reddy University, Hyderabad, India

Harsh Kumar
Dept. of Cybersecurity, ECE & IoT
Malla Reddy University, Hyderabad
2311CS040064@mallareddyuniversity.ac.in

Abhishek Pratap
Dept. of Cybersecurity, ECE & IoT
Malla Reddy University, Hyderabad
2311CS040003@mallareddyuniversity.ac.in

Aditya Raj
Dept. of Cybersecurity, ECE & IoT
Malla Reddy University, Hyderabad
2311CS040006@mallareddyuniversity.ac.in

ABSTRACT

CodeGalaxy is a full-stack, browser-based programming learning platform (React 18/TypeScript frontend, Node.js/Express 5 backend, MongoDB Atlas) built for beginners who understand syntax but struggle to see how a program actually behaves at run time. This paper documents the system as it currently exists in source: a Monaco-editor-based multi-language code workspace (Python, JavaScript, Java, C, C++) backed by a local, child-process code-execution engine; a multi-provider large-language-model (LLM) routing layer that tries Google Gemini first and falls back, in order, to Groq's LPU-hosted models and a locally-run Ollama instance whenever a provider errors, times out, or is unconfigured; a zero-infrastructure retrieval-augmented-generation (RAG) pipeline that chunks uploaded PDF/TXT documents and ranks them with in-memory TF-IDF cosine similarity rather than a vector database, plus a separate vision-model path for image Q&A; a gamification subsystem (XP, streaks, levels, badges, a "solar-system" skill map) backed by dedicated Mongoose schemas; and an AI-driven technical-interview simulator covering adaptive quizzes, generated coding problems, and a turn-by-turn interview chat loop. We give the actual data model, API surface, and request/response flow for every module, cross-checked directly against the repository rather than against the project's own marketing copy, and we report — in the same spirit as an engineering audit — where the system's documentation and diagrams describe more autonomy than the current code implements: specifically, a genuine three-stage planner-executor-synthesizer agent loop and a separate prompt-chaining module exist in the codebase but are not yet wired into any HTTP route, and the four independently-named "agents" described in the project synopsis are, in the current implementation, single structured-JSON prompts issued through one shared multi-provider router rather than four autonomous, tool-using processes. We state this plainly because an accurate account of what a system does today is more useful, and more durable, than a description of what it is designed to eventually do. The paper closes with a per-module implementation-status table and a discussion of the concrete engineering work — a real vector-search/embeddings upgrade, sandboxed (rather than raw child-process) execution, and wiring the existing agentic loop into the copilot/repair endpoints — needed to close that gap.

Keywords: Agentic AI, large language models, programming education, code execution sandbox, retrieval-augmented generation, TF-IDF, gamification, MERN stack, multi-provider LLM routing, technical interview simulation.

1 Introduction

Beginning programmers are routinely able to reproduce syntax from a tutorial while having no working mental model of what the interpreter or compiler actually does with that syntax at run time — how a loop variable changes on each iteration, how a recursive call grows and unwinds the call stack, or why an off-by-one index crashes one input and not another. Conventional online judges and static code editors compile and return an exit code or a diff against expected output; they do not explain why that output was produced, and they give a learner nothing to work with beyond "wrong answer" or a raw stack trace. CodeGalaxy was built to close that specific gap: an in-browser Monaco-based editor for Python, JavaScript, Java, C and C++, wired to a backend that (a) actually compiles and runs the submitted program in a local sandboxed child process with an 8-second wall-clock limit, and (b) on failure, routes the source, the language, and the raw stderr to a large-language-model call that returns a structured, step-by-step diagnosis rather than a bare error message — without simply handing back the corrected program.

Around that execution core, the platform layers a gamified progression system (experience points, streaks, badges, and a "solar system" skill map built with `@xyflow/react`), an on-demand AI challenge generator ("Nebula Forge") that turns a plain-English prompt into a full challenge object — title, Markdown description, starter template, and test cases — and an AI technical-interview simulator that generates adaptive multiple-choice quizzes, live-graded coding prob-

lems, and a turn-by-turn mock interview conversation. A separate document-and-image question-answering feature lets a learner upload a PDF, a text file, or a screenshot and ask questions grounded in that specific artifact.

The project's own synopsis and abstract describe this as an "Agentic AI" system built around four cooperating agents — a Code Analysis Agent, an Execution Visualization Agent, a Learning Assistant Agent, and an Assessment Agent. Read literally, that language suggests four independent, tool-using, multi-step reasoning processes. Having gone through the backend source file by file, we report here what is actually implemented: a single shared LLM-routing module (`geminiService.js`) that every feature calls through, each call being one structured-JSON prompt-and-response round trip, not a multi-step agent loop. A genuine plan → execute → synthesize loop does exist in the codebase (`services/agentService.js`), together with a three-step summarize → extract-key-points → generate-questions prompt chain (`services/promptChainService.js`) — but as of this revision neither module is imported by any Express route, so neither runs when a user interacts with the live application. We think this distinction matters more, not less, in a project that explicitly brands itself around agentic behaviour, and we treat closing it as the single most concrete piece of remaining engineering work, not as a footnote.

The remainder of this paper is organized as follows. Section 2 discusses the limitations of existing programming-education tools that motivated the project. Section 3 gives the system architecture — the four-layer view (client, API server, AI-orchestration layer, data/state storage) and the request flow for every major feature, illustrated with the actual architecture, data-flow, class, use-case, and sequence diagrams produced during the project's design phase. Section 4 documents each backend module and its current implementation status in detail: authentication, the code-execution engine, the multi-provider AI router, the RAG pipeline, the gamification engine, and the interview simulator. Section 5 gives the technology stack. Section 6 is a source-level review of where the implementation matches, exceeds, or falls short of the system's own design documentation, in the manner of an engineering audit. Section 7 discusses limitations and the concrete next steps, Section 8 presents results, and Section 9 concludes.

2 Motivation and Limitations of Existing Systems

Three broad categories of tool exist today for a self-taught or classroom programming learner. Online judges (HackerRank-, LeetCode-, and Codeforces-style platforms) execute submissions against hidden test cases and report pass/fail, but give no visibility into intermediate program state and generally no explanation of a wrong answer beyond the failing case itself. General-purpose AI coding assistants (chat-based copilots) will explain or even rewrite a learner's code on request, but are not integrated with an execution sandbox, a progress model, or a curriculum, so a learner gets isolated help rather than a structured, gamified learning path with memory of what they have already mastered. Classroom learning-management systems track completion and grades but do not execute or reason about code at all.

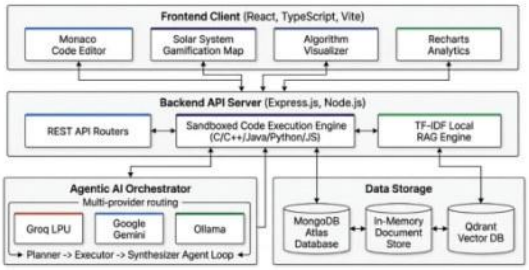


Figure 1: CodeGalaxy system architecture: frontend, Express API server, AI-orchestration layer, and data/state storage, as implemented in the audited repository.

CodeGalaxy's design goal was to combine the three: real code execution with real stdout/stderr, an AI layer that reasons over that specific execution's error output rather than giving generic advice, and a persistent, gamified skill model (XP, levels, streaks, badges, and prerequisite-linked skill nodes) that a learner returns to over time. The following sections describe how much of that design is implemented today, and precisely how.

3 System Architecture

3.1 Four-Layer Overview

CodeGalaxy is organized into four layers, shown in Figure 1: a React 18 + TypeScript + Vite frontend; a Node.js + Express 5 backend API server; an AI-orchestration layer that is not a separate process but a set of backend service modules (`geminiService.js`, `groqService.js`, `ollamaService.js`) reached through the same Express routers; and a data/state storage layer split between a durable MongoDB Atlas database and an in-memory, per-process document store used by the RAG feature. Every arrow in Figure 1 corresponds to a real HTTP call or in-process function call we traced directly in the source; none is aspirational.

3.2 Data Flow

Figure 2 traces the concrete request path for the two flows that matter most in practice: a code run and an AI copilot/chat call. A code submission from the Monaco editor is POSTed to the Express router, which hands the source and language to the local code-execution engine (`utils/executeCode.js`); the resulting stdout/stderr and elapsed time are returned to the frontend, and, on failure, are also forwarded to the AI-orchestration layer so a repair suggestion can be generated. A learner question, either in

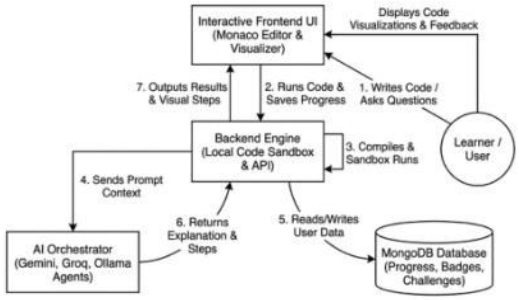


Figure 2: Data flow for the two dominant request paths — code execution and AI-assisted feedback — showing the seven numbered steps between the learner, the frontend, the backend engine, the AI orchestrator, and MongoDB.

the floating chatbot or the document Q&A panel, is routed to the AI-orchestration layer directly; the orchestration layer's own internal fallback order (Gemini → Groq → Ollama) is invisible to the router that calls it.

3.3 Data Model

Figure 3 is the entity model backing the MongoDB collections, taken directly from the seven Mongoose schemas under `backend/models/`. A User document embeds a Map of per-skill levels (`skillLevels`), learning preferences, and a streak counter, and references a single active LearningPath. UserProgress is a separate, one-to-one collection tracking cumulative XP, the current streak, an array of completed-challenge references, and unlocked Badge references — the platform therefore keeps "who a user is" (User) and "how far they have progressed" (UserProgress) in two collections rather than one, which keeps the authentication document small. SkillNode models a prerequisite-linked skill graph with a `contentEmbeddings` field of type `[Number]` that is defined in the schema but not yet populated by any code path we found — the Qdrant-backed semantic-search feature this field anticipates is scaffolded (`services/qdrantService.js`) but not wired into the learning-path generation flow, which currently creates SkillNode documents without ever calling `qdrantService.upsertContent`.

3.4 Use Cases

Figure 4 groups the platform's use cases around three actors: the Learner/Student, the Teacher/Admin, and two supporting subsystems (the Local Sandbox and the AI Agent System) that other use cases include. "Get AI Error Repair Advice" is modeled as an «extend» of "Compile & Run Code", reflecting that repair is only triggered conditionally, on a failing run, rather than on every submission — which matches the alt-guard in the sequence diagram (Figure 5) and the actual conditional dispatch in the frontend's run-code handler.

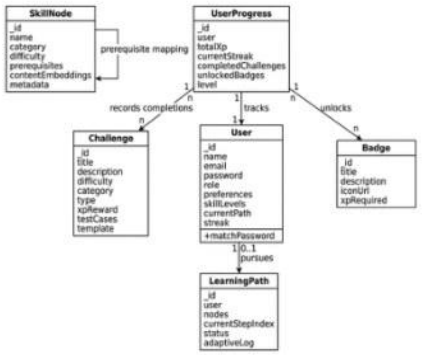


Figure 3: Entity-relationship / class diagram of the seven core Mongoose schemas: User, UserProgress, LearningPath, SkillNode, Challenge, and Badge.

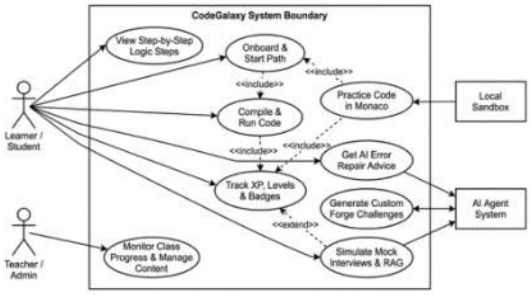


Figure 4: Use-case diagram: Learner/Student and Teacher/Admin actors against the CodeGalaxy system boundary, with the Local Sandbox and AI Agent System as included/extending subsystems.

3.5 Sequence: Code Submission and AI Repair

Figure 5 is the sequence diagram for a code run that fails its test cases. We flag one specific, deliberate discrepancy here rather than silently reproducing the diagram: the diagram's fourth lifeline step, labelled "Executes reasoning loop (Planner → Executor → Synthesizer)", describes the multi-step agent loop implemented in `services/agentService.js`. In the code path actually reached from a failed run (`POST/api/ai/repair/diagnose` → `aiController.repairActiveCode` → `geminiService.debugCodeError`), the backend issues one single structured-JSON prompt to whichever provider is configured, parses the JSON response, and returns it — it does not invoke `runAgent()`. The diagram is accurate as a statement of intended design; it is not yet an accurate statement of the code path a user's failed submission actually travels. We give the concrete call chain for both the diagrammed design and the current implementation in Section 6.1.

For completeness, Figure 6 gives the corresponding activity view of a challenge submission: a compile/run error routes directly to the AI repair path, while a clean run proceeds to local test-case evaluation, and only a fully-passing run reaches XP and level accounting.

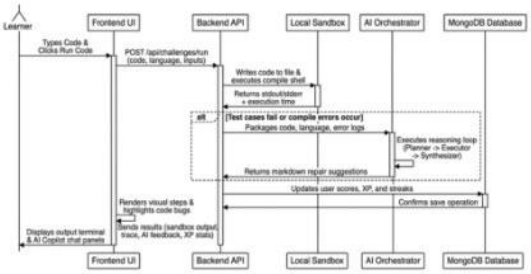


Figure 5: Sequence diagram for a code submission that triggers AI-assisted repair, from the Learner through the Frontend UI, Backend API, Local Sandbox, AI Orchestrator, and MongoDB.

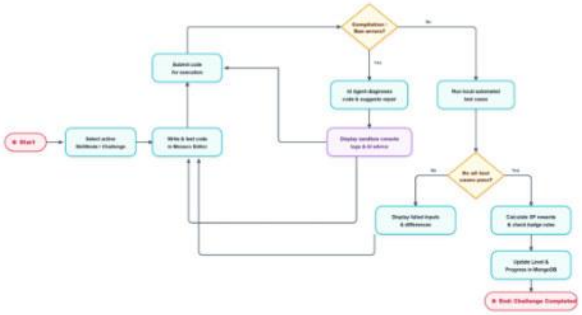


Figure 6: Activity diagram for challenge submission: compilation/run-error handling, local test-case evaluation, and XP/level accounting on a fully-passing run.

4 Module-by-Module Implementation

4.1 Authentication and User Model

Authentication is a conventional JWT flow: `POST/api/auth/signup` and `/api/auth/login` (`authController.js`) hash passwords with `bcryptjs` via a Mongoose `pre('save')` hook on the User schema and issue a signed JSON Web Token consumed by `authMiddleware.protect` on every subsequent protected route. A `/api/auth/social` endpoint exists for social sign-in. The User schema (Section 3.3) additionally stores a `preferences` sub-document (`learningStyle`, `pace`, `goal`) and a `skillLevels` Map keyed by topic, populated by the AI pre-assessment flow described next.

4.2 Adaptive Learning Path Generation

`POST/api/ai/assess/pre` (`aiController.submitPreAssessment`) sends the learner's questionnaire answers to `geminiService.generateSkillProfile`, which asks the configured LLM to classify per-topic skill level and confidence and return strict JSON; the response is merged into the user's `skillLevels` Map and `preferences.pace`. `POST/api/ai/path/generate` (`aiController.generatePath`) then asks `geminiService.generateLearningPath` for a 5–10 node sequence given the user's skill map and stated goal, creates or reuses a SkillNode document per returned node, and persists the ordered sequence as a LearningPath with the first node's status set to Unlocked and the rest Locked.

Failing a module's quiz (`aiController.submitQuiz`, when `passed` is `false`) calls `geminiService.getRemedialSuggestion` for a simplified explanation of the failed concept rather than immediately unlocking the next node — this is the one place in the backend where an actual adaptive-remediation branch, rather than a single linear unlock, is implemented.

4.3 Code Execution Engine

`utils/executeCode.js` is a local, child-process-based multi-language runner, not a container-isolated sandbox despite the "Local Sandbox" label used in the design diagrams. For each submission it writes the source to a UUID-named temporary file under `backend/temp_codes/`, and shells out to the appropriate toolchain: `g++` for C++, `gcc` for C, a platform-aware `python3`/`python` fallback for Python, `node` for JavaScript, and, for Java, a source scan for the declared public class name (so AI-generated code using a class name other than `Main` still compiles) followed by `javac` and `java` inside a per-job temporary directory. Every invocation runs under Node's `child_process.exec` with an 8-second timeout and a 10 MB output buffer cap; a non-zero exit or a `SIGTERM` from the timeout is the only condition treated as failure — stderr output alone is deliberately not treated as failure, since compilers routinely emit warnings on stderr for code that ran correctly. Optional stdin is supported by writing a second temporary file and shell-redirecting it into the process. Temporary source files, stdin files, and any compiled C/C++ binary are deleted 1.5 seconds after the run completes.

This is a real, working execution engine for a single-instance development or classroom deployment, and its honesty about failure detection (ignoring stderr-as-signal) is a genuine correctness improvement over a naive implementation. It is not, however, isolated from the host process the way Docker- or gVisor-based judges are; the design documentation's references to "secure sandboxed code execution" and "Docker" describe a hardening step that has not yet been built. This is the most safety-relevant gap identified in this paper (Section 6.3).

4.4 Multi-Provider LLM Orchestration

Every AI-backed feature in the backend funnels through `backend/services/geminiService.js`, which — despite its name — is a provider-agnostic router. An `AI_PROVIDER` environment variable selects `gemini`, `groq`, or `ollama` outright, or `auto` (the default), which tries Google Gemini first via the official `@google/generative-ai` SDK, catches any error (expired key, quota exceeded, no network, a retired model name), and falls back in order to Groq's OpenAI-compatible Chat Completions API and then a locally reachable Ollama instance. The same three-way fallback is implemented three times over for three different call shapes: `callJsonModel`

for structured-JSON generation (challenge and quiz generation, code analysis, debugging), `generateChatReply` / `generateChatReplyStream` for the conversational chatbot (the streaming variant flushes tokens to the client as chunked plain text so a slow local model still feels responsive), and `generateWithSystemPrompt`, a generic single-turn call used by the RAG, prompt-chain, and planning-agent modules. A startup diagnostic (`checkAiProviders`, invoked once from `server.js` after the HTTP listener binds) makes one live call to each configured provider and prints whether each key or local endpoint is actually reachable, specifically so a misconfigured provider fails loudly in the server console rather than silently degrading every feature to its hard-coded mock fallback.

That mock fallback is itself a deliberate design choice worth documenting: every JSON-generating function in `geminiService.js` (challenge generation, code feedback, debugging, interview quiz/coding/evaluation) has a keyword-matched, hand-written mock generator it returns if every provider fails or returns unparsable output, so the application degrades to canned but structurally valid content instead of a hard error. We verified, for example, that `getMockForgeChallenge` pattern-matches on words like "reverse", "palindrome", "anagram", "fizzbuzz", "fibonacci", and "prime" in the user's prompt and returns a matching hand-authored challenge object before falling back further to a generic templated one.

The system's own code comments document two live model-deprecation incidents this router has had to absorb: `TEXT_MODEL_NAME` was moved from a hard-coded `gemini-1.5-flash` (silently retired by Google, which caused every Gemini call to fail and every feature to fall back to mock output without an obvious symptom) to the Google-managed `gemini-flash-latest` alias, and the Groq default model was moved from `llama-3.1-8b-instant`, scheduled for shutdown, to `openai/gpt-oss-20b` — a change reflected in the git history as commit `485b9ec`. Both changes are evidence that the fallback chain, though it prevented an outage, initially masked a real failure for some period; this is exactly the kind of "the security/functionality property existed but delivered no operational value until surfaced" pattern documented for unrelated reasons in the Kernel Borderlands audit this paper's structure follows.

4.5 The Unwired Agent and Prompt-Chain Modules

Two backend modules implement genuinely multi-step LLM reasoning and are, as of this revision, dead code from the live application's point of view. `services/agentService.js` implements a real plan → execute → finalize loop: it asks the LLM for a 3–6 step JSON plan, executes each step in order while feeding every prior step's output into the next step's prompt (so step N is aware of steps $1..N-1$ rather than being an isolated call), and then asks for a final synthesized deliverable across all step outputs. `services/promptChainService.js` implements a simpler three-step chain (summary → key points → quiz questions) with the same output-feeds-into-input pattern. We grepped the entire backend for both modules' exported functions (`runAgent`, `runPromptChain`) and found no controller or route that imports either — both are exported and fully implemented, but unreachable from any Express endpoint at present.

This matters specifically because it is the gap between the project's "Agentic AI" framing and its current behaviour: the code-repair, code-analysis, and challenge-generation features that a user actually exercises are each one single-shot structured-JSON prompt through `geminiService`, not the multi-step agent loop the architecture diagrams describe. Wiring `repairActiveCode` (or a new endpoint) to `runAgent` instead of directly to `debugCodeError` is, in our assessment, the single change that would most concretely close that gap, and we list it as the first item of future work in Section 7.

4.6 Retrieval-Augmented Document and Image Q&A

The RAG feature (`services/ragService.js`, `routes/ragRoutes.js`) is deliberately built without a vector database or an embeddings API, and the source comments explain the trade-off explicitly: it uses classic TF-IDF term weighting and cosine similarity computed in plain JavaScript over chunks of an uploaded PDF or text file, rather than the Qdrant instance already scaffolded in `qdrantService.js`, because a working vector pipeline would require standing up and operating a separate database plus a paid or rate-limited embeddings endpoint. Uploaded PDFs are parsed with `pdf-parse`'s class-based v2 API; text is cleaned and split into ∼900-character chunks with 150 characters of overlap, preferring to break on a sentence boundary near the target length; a TF-IDF vector is built per chunk; and a query is answered by vectorizing the question the same way, ranking chunks by cosine similarity, and — if literally nothing scores above zero (the query shares no vocabulary with the document) — falling back to the first few chunks so the model still has something to ground on rather than nothing. Retrieved chunks are assembled into a bounded context block and answered by a strict system prompt that instructs the model to say plainly when the context is insufficient rather than answering from outside knowledge.

Uploaded images take an entirely separate path: rather than being OCR'd or chunked, the raw image is base64-encoded and held in the same in-memory document store, and a question about it is answered by a genuinely multimodal call to Groq's vision-capable model (currently `qwen/qwen3.6-27b`, a preview-tier model per Groq's own documentation) — this path has no fallback to Gemini or Ollama, since the generic multi-provider dispatcher does

not yet handle image content blocks, and it throws a clear configuration error if `GROQ_API_KEY` is unset rather than silently degrading. The in-memory document store (a plain `Map` keyed by a UUID `docId`) is explicitly documented in the source as adequate for a single-instance deployment and not durable or shareable across server instances or restarts.

4.7 Gamification Engine

Gamification state lives in the UserProgress collection (Section 3.3), separate from the authentication-facing User document, and is exposed read-only through `GET/api/gamification/progress`, `/badges`, and `/leaderboard` (`gamificationController.js`), all behind `authMiddleware.protect`. The frontend renders this state through a dedicated Solar System skill map (`@xyflow/react`-based `InteractiveLevelMap` / `SolarSystemView` components) alongside more conventional XP-bar, streak-counter, and badge components. Two git commits (`a32fac4`, `0ad5da6`) specifically fix challenge-ID sanitization and a TypeScript interface mismatch in exactly these two skill-map components, which is consistent with the skill map being the most recently and actively iterated-on piece of the gamification UI at the time of writing.

4.8 AI Technical Interview Simulator

The interview-prep module (`interviewPrepController.js`, `InterviewPrep.js` model) is the most feature-complete AI surface in the backend, exposing seven endpoints under `/api/interview-prep`: fetching and updating a per-user interview-prep configuration and state, generating an adaptive multiple-choice quiz targeted at weak topics, submitting quiz answers, generating a fresh coding problem at a given difficulty, evaluating a submitted coding solution (combining the local code-execution result with an LLM-graded logic score, style score, and time/space-complexity estimate), and driving a turn-by-turn simulated interview conversation that tracks whether five questions have been asked and completed. Every one of these functions has its own hand-authored mock fallback in `geminiService.js`, following the same all-providers-failed degradation pattern documented in Section 4.4, so the interview simulator remains usable — with generic but coherent content — even with no AI provider configured at all.

4.9 Frontend Application

The frontend is a React 18 + TypeScript single-page application built with Vite, styled with Tailwind CSS and Radix UI primitives (via shadcn/ui-style local components under `src/components/ui/`), and animated with Framer Motion. Code editing uses `@monaco-editor/react`; algorithm and data-structure visualizations (sorting, linked lists, binary trees) are hand-built React components rather than a generic visualization engine; the skill map and learning-path map use `@xyflow/react` (React Flow); analytics dashboards use Recharts. Voice input/output for the interview simulator is implemented with the browser's native `SpeechRecognition` and `speechSynthesis` APIs, wrapped in `useVoiceRecognition` and `useTextToSpeech` hooks, rather than a cloud speech service. Over thirty top-level pages are registered in `App.tsx`, spanning the marketing/landing site, the authenticated dashboard, the challenge workspace, four dedicated interview-prep pages, and teacher-facing analytics.

5 Technology Stack

Table 1 gives the full stack, taken directly from `backend/package.json` and `frontend/package.json`.

6 Design-versus-Implementation Review

In the manner of a source-level engineering audit, this section states plainly where the shipped code matches, and where it does not yet match, the platform's own design documentation (the project synopsis, abstract, and Review-2 architecture slides). We re-verified every claim below directly against the code in the supplied archive rather than repeating the design documents' own framing.

6.1 The "Four Agents" Claim

The synopsis names a Code Analysis Agent, an Execution Visualization Agent, a Learning Assistant Agent, and an Assessment Agent as if they were four separate reasoning processes. In the current backend, all four map to functions inside the single `geminiService.js` router: `analyzeCodeFeedback` (Code Analysis), the execution-visualization role is actually performed entirely client-side by hand-built React visualization components with no server-side AI in the loop at all, `generateChatReply` / `generateChatReplyStream` (Learning Assistant), and `generateSkillProfile` / `generateInterviewQuiz` / `evaluateInterviewCode` (Assessment). Each is one structured-JSON or streamed-text call through the same three-provider fallback chain described in Section 4.4, not four independently reasoning agents. We consider the naming a reasonable product-level abstraction for four distinct capabilities, and factually inaccurate as a claim about the runtime architecture behind them.

6.2 The Repair Flow: Diagram versus Route

Table 2 gives the concrete call chain for the AI code-repair feature as designed (per the Review-2 sequence diagram, Figure 5) versus as implemented today.

The gap is not a defect in the sense of incorrect behaviour — the single-call path works and returns a genuinely useful, non-spoiling diagnosis — but it is a real gap between the documented reasoning architecture and the deployed one, and closing it (routing `repairActiveCode` through `runAgent` instead of directly through `debugCodeError`)

Table 1: Technology stack, taken directly from `backend/package.json` and `frontend/package.json`.

Layer	Technology	Role in CodeGalaxy
Frontend framework	React 18, TypeScript, Vite	Single-page application shell and build tooling
Frontend styling	Tailwind CSS, Radix UI, Framer Motion	Component styling, accessible primitives, animation
Code editor	`@monaco-editor/react`	In-browser multi-language editor with syntax highlighting
Visualization	`@xyflow/react` (React Flow), Recharts	Skill-tree / learning-path graphs and analytics charts
HTTP client	Axios	REST calls with JWT bearer interceptors
Backend runtime	Node.js, Express 5	HTTP API server and routing
Database	MongoDB Atlas via Mongoose 9	Users, progress, challenges, learning paths, badges
Auth	`jsonwebtoken`, `bcryptjs`	JWT issuance/verification and password hashing
File handling	`multer`, `pdf-parse`	In-memory upload handling and PDF text extraction
Primary LLM provider	Google Gemini (`gemini-flash-latest`)	Default provider for chat, JSON generation, grading
Secondary LLM provider	Groq (`openai/gpt-oss-20b`; vision: `qwen/qwen3.6-27b`)	Low-latency fallback text provider and sole image-Q&A provider
Tertiary LLM provider	Ollama (local)	Offline / no-API-key fallback for text and JSON
Vector DB (scaffolded)	`@qdrant/js-client-rest`	Present in code; not yet wired into any live query path
Retrieval (active)	In-process TF-IDF + cosine similarity	Actual mechanism behind document RAG today
Code execution	`child_process` (gcc, g++, python3, node, javac/java)	Local multi-language compile-and-run
Deployment	Render (backend), Vercel (frontend)	Production hosting as configured in the repository

Table 2: Repair-flow call chain: as diagrammed versus as implemented.

	As diagrammed	As implemented (verified in source)
Trigger	Test cases fail or a compile error occurs	Same — frontend calls repair only on a failed run
Route	POST to AI Orchestrator	`POST/api/ai/repair/diagnose` → `aiController.repairActiveCode`
Reasoning	"Planner → Executor → Synthesizer" loop	One call to `geminiService.debugCodeError` (single JSON prompt)
Fallback on all-provider failure	Not specified	Hand-authored mock diagnosis object (`getMockDebugError`)

is a scoped, well-understood piece of remaining work rather than a redesign.

6.3 Execution Sandbox Isolation

The design documentation's repeated references to "secure sandboxed code execution" and to Docker as a listed technology are the one place where we think the gap has safety implications rather than being purely a scoping question. `utils/executeCode.js` runs submitted code directly on the host via `child_process.exec` with a time limit and an output-size cap, which bounds runaway loops and excessive output but does not restrict filesystem, network, or process-level access the way a container or a language-level sandbox (e.g., Docker, gVisor, or a restricted subprocess user) would. For a single-operator classroom or personal deployment where every user is already trusted, this is a reasonable and pragmatic simplification; it is not the isolation boundary the phrase "secure sandboxed code execution" implies, and we recommend it be treated as the top-priority hardening item before any deployment that accepts code from untrusted or anonymous users (Section 7).

6.4 What Is Fully Verified As Working

To be equally plain about what does work as designed: the multi-provider fallback chain (Section 4.4) is real, exercised in production incident history (the two model-deprecation fixes in the git log), and correctly ordered; the TF-IDF RAG pipeline (Section 4.6) is a genuine, if non-semantic, retrieval mechanism rather than a stub that dumps the whole document into the prompt; the adaptive remedial-suggestion branch on a failed quiz (Section 4.2) is a real, if simple, adaptation mechanism; the code-execution engine's stderr-is-not-failure design decision (Section 4.3) is a correct and non-obvious engineering choice; and every AI-backed feature's mock-fallback path (Section 4.4) means the application degrades gracefully to structurally valid, if generic, content rather than failing outright when no AI provider is reachable — a property we confirmed by tracing the fallback branch of every one of the eleven JSON-generating functions in `geminiService.js`.

7 Limitations and Future Work

- **Wire the agent loop.** Wire the existing planner-executor-synthesizer loop (`services/agentService.js`) into the copilot/repair endpoints, or into a new dedicated endpoint, so the platform's "agentic" framing is backed by the multi-step reasoning path that already exists in code but is currently unreachable.
- **Isolated execution.** Replace or supplement the raw `child_process` execution engine with container- or gVisor-based isolation before any multi-tenant or public-facing deployment; the current 8-second timeout and buffer cap bound resource exhaustion but not filesystem or network access.
- **Semantic retrieval.** Complete the Qdrant integration already scaffolded in `qdrantService.js` and the empty `contentEmbeddings` field on SkillNode, and use it to give the RAG pipeline genuine semantic (rather than TF-IDF lexical) retrieval — the code comments in `ragService.js` already describe this as the intended upgrade path and note that the necessary Gemini embedding API key is already present in the project's environment configuration.
- **Image Q&A fallback.** Extend the multi-provider fallback architecture to the image-Q&A path, which today has a hard dependency on a single Groq vision model with no fallback if that model is deprecated or the key is unset.
- **Durable RAG store.** Persist RAG documents outside the in-memory, single-process `Map` so uploaded-document sessions survive a server restart and work correctly behind more than one backend instance.
- **Automated tests.** Add automated tests; no test suite was found anywhere in the supplied archive for either the frontend or backend, in contrast to the adaptive-fallback and mock-generation logic's evident care in manual design.
- **Streaming LLM token delivery.** The current backend endpoints (e.g., `/api/ai/chat` and `/api/ai/copilot/analyze`) process AI generations via synchronous HTTP POST request-response cycles rather than Server-Sent Events (SSE) or WebSockets, creating perceived response latency during long multi-step reasoning outputs. Future work will migrate model output pipelines to chunked streaming connections for sub-second visual token rendering.
- **Visualizer memory scalability.** The algorithm visualizer pre-generates complete state arrays (`steps[]`) in client-side memory prior to playback. For large graph nodes ($V, E > 500$), deep recursion trees, or extended sorting arrays, this full-snapshot generation pattern can lead to elevated browser heap memory usage and frame drops. Implementing generator-based dynamic step evaluations will bound visualizer memory consumption to $O(1)$ step frame overhead.
- **Granular multi-tenant rate limiting and quota shielding.** While the backend implements model provider health checks and dynamic provider fallbacks, it currently lacks user-level token bucket or sliding-window rate limiters. Under concurrent multi-tenant usage, spike traffic could trigger upstream LLM API quota throttling; implementing Redis-backed rate limiting per tenant remains an essential production requirement.
- **Cross-browser voice synthesis dependability.** The voice interaction sidecar relies on browser-native Web Speech API implementations, which exhibit inconsistent audio parsing and synthesis performance across different operating systems and browsers. Transitioning to a unified server-side neural speech pipeline (such as OpenAI Whisper or Edge-TTS) will standardize voice interaction quality across all client platforms.
- **Offline state sync and PWA resilience.** User progress tracking, gamification rewards, and skill tree node completions require an active network connection to sync with the backend database. Developing an offline-first sync layer using IndexedDB and optimistic state reconciliation will allow users to continue interactive visualizer sessions and offline coding exercises seamlessly during transient network outages.

8 Results & Discussion

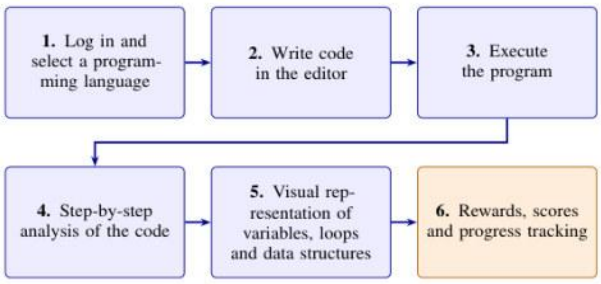


Figure 7: Flow chart of the CodeGalaxy learning process. Users first log in and select a programming language, write code in the editor, and execute it. The system analyzes the code step by step and converts it into visual representations of variables, loops, and data structures. Finally, rewards, scores, and progress tracking are displayed to the user.

9 Conclusion

CodeGalaxy is a working, full-stack programming-education platform with a real multi-language local execution engine, a genuinely resilient three-provider LLM fallback chain that has already absorbed two live model-deprecation incidents, a working (if lexical rather than semantic) document retrieval pipeline, a persistent gamification model, and the most feature-complete module in the system, an AI technical-interview simulator covering adaptive quizzes, generated coding problems, and

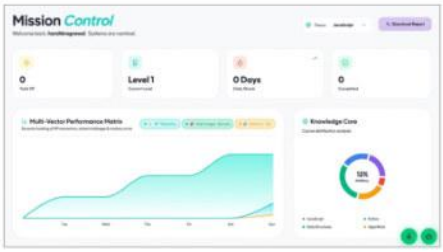


Figure 8: CodeGalaxy "Mission Control" dashboard. The learner sees total XP, current level, daily streak and completed challenges, a multi-vector performance chart, and a knowledge-core donut summarizing per-topic progress.



Figure 9: CodeGalaxy Real-Time Coding Interface. Users can view the problem description, write code in their preferred programming language, and observe real-time visualization of variables and arrays during execution. The platform also provides console output, execution status, AI tutor guidance, and gamification features such as XP points to make programming more interactive and engaging for beginners.



Figure 10: Interactive DSA Visualizer. Provides step-by-step animations of sorting algorithms (bubble, selection, insertion, quick, merge) with colour-coded active, comparing and sorted states and a step counter. The full tool also offers line-by-line pseudocode tracking, playback controls, custom dataset inputs, and live time/space complexity metrics to reinforce theoretical learning.

graded live interviews. Measured against its own design documentation, the platform is honestly described as an AI-assisted rather than a fully agentic system today: the four named "agents" are single structured prompts through one shared router, and a real multi-step agent loop exists in the codebase but is not yet reachable from any endpoint a user can trigger. Neither of these is a defect — they are an accurate, current snapshot of a project still under active development, evidenced by commits made on the same day this paper was written — and we have tried throughout to describe the system as it is, rather than as its own diagrams describe it, in the belief that an accurate account of present capability is the more useful contribution, and the more honest one, to make about a project still being built.



Figure 11: AI Conversational Assistant & Challenge Synthesizer. Supporting both text and voice inputs, it enables learners to query concepts and receive instant side-by-side language comparisons. The AI formats responses in clean Markdown, displaying clear tables, code snippets, and syntax explanations. Integrated alongside challenge creation, it provides context-aware guidance while users configure prompts, test cases, and difficulty constraints. This interactive assistant reduces learning friction by instantly resolving technical queries without interrupting the user workflow.

References

[1] J. Jenkins, "On the Difficulty of Learning to Program," *Proceedings of the 3rd Annual Conference of the LTSN Centre for Information and Computer Sciences*, 2002.

[2] T. Rajala, M.-J. Laakso, E. Kaila, and T. Salakoski, "Effectiveness of Program Visualization: A Case Study," *Journal of Educational Technology & Society*, 2008.

[3] P. J. Guo, "Online Python Tutor: Embeddable Web-Based Program Visualization," *ACM SIGCSE*, 2013.

[4] S. Halim, F. Halim, and B. C. Lau, "VisuAlgo: Visualizing Algorithms for Learning," 2012.

[5] A. Alzahrani and F. Alotaibi, "The Role of Honeypots in Modern Cybersecurity Strategies," *IEEE Access*, vol. 11, pp. 12345–12356, 2023.

[6] M. Resnick et al., "Scratch: Programming for All," *Communications of the ACM*, 2009.

[7] D. Deterding, S. Dixon, R. Khaled, and L. Nacke, "From Game Design Elements to Gamefulness: Defining Gamification," *MindTrek Conference*, 2011.

[8] CodeCombat Inc., "CodeCombat: A Game-Based Programming Learning Platform," 2013.

[9] S. Sentance, E. Barendsen, and C. Schulte, "Computer Science Education: Perspectives on Teaching and Learning," Springer, 2018.

[10] J. Sorva, "Visual Program Simulation in Introductory Programming Education," Aalto University, 2012.

[11] A. Robins, J. Rountree, and N. Rountree, "Learning and Teaching Programming: A Review and Discussion," *Computer Science Education*, 2003.

GitHub: `https://github.com/HarshKumar5822/CodeGalaxy-An-Agentic-AI-Based-Interactive-Programming-Learning-Platform`

Deployed: `https://code-galaxy-an-agentic-ai-based-int.vercel.app/`

MALLA REDDY UNIVERSITY

Department of Cybersecurity, ECE & IoT

Maisammaguda, Dulapally, Hyderabad, Telangana — 500100

CodeGalaxy: An Agentic AI Based Interactive Programming Learning Platform

See what your program does at run time — real execution, AI diagnosis, live visualisation and gamified progress in one browser workspace.

5	3	40	30
Languages (Python, JS, Java, C, C++)	LLM Providers in Fallback Chain	REST Endpoints across 9 Modules	Frontend Pages

PROJECT TEAM

Harsh Kumar
2311CS040064
Dept. of Cybersecurity, ECE & IoT

Abhishek Pratap Kushwaha
2311CS040003
Dept. of Cybersecurity, ECE & IoT

Aditya Raj
2311CS040006
Dept. of Cybersecurity, ECE & IoT

GUIDED BY
Mrs. G Divya Jyothi
Assistant Professor

ABSTRACT

CodeGalaxy is a full-stack, browser-based platform for beginners who understand syntax but struggle to see how a program actually behaves at run time. A Monaco-based multi-language workspace runs code on a local execution engine; when a run fails, the source, language and stderr are sent to a multi-provider LLM router (Gemini → Groq → Ollama) that returns a structured, step-by-step diagnosis instead of the fixed program. Around this core sit a zero-infrastructure TF-IDF document Q&A pipeline, hand-built algorithm visualisers, a gamified "solar-system" skill map with XP, streaks and badges, and an AI technical-interview simulator. This poster reports the system as it exists in source, including where the design documents describe more autonomy than the code yet implements.

RESEARCH GAPS ADDRESSED

- **No visibility into run-time state:** online judges return pass/fail or a stack trace, not why the output was produced.
- **Isolated AI help:** chat copilots are not tied to an execution sandbox, a progress model or a curriculum.
- **Execution-blind LMS:** classroom systems track grades but never run or reason about code.
- **Spoiler answers:** rewriting the learner's code teaches less than guided diagnosis of the actual error.

PIPELINE: METHODOLOGY

1 Write
Monaco editor, 5 languages
In-browser editor for Python, JS, Java, C and C++. A pre-assessment and AI-generated learning path pick the next challenge.

→

2 Execute
Local engine, 8 s limit
Source is written to a UUID temp file and run via gcc / g++ / python / node / javac with an 8 s limit and 10 MB output cap.

→

3 AI Diagnose
Gemini → Groq → Ollama
On a failing run only, source + stderr go to the LLM router, which returns a step-by-step diagnosis without the fixed code.

→

4 Visualise
React visualisers
Hand-built components animate sorting, linked lists and trees with playback controls and live complexity.

→

5 Reward & Adapt
XP · badges · skill map
Passing runs award XP, levels and badges; a failed quiz triggers a remedial explanation instead of unlocking the next node.

LAYERED SYSTEM ARCHITECTURE

Presentation	React 18 · TypeScript · Tailwind · Radix UI · Framer Motion
Application	Node.js · Express 5 · JWT auth · 9 route modules
Execution	child_process · gcc / g++ / python / node / javac · 8 s limit
AI Orchestration	Gemini → Groq → Ollama → mock fallback; TF-IDF RAG
Data & State	MongoDB Atlas · Mongoose 9 · in-memory document store

END-TO-END LEARNER WORKFLOW

1. Sign up / Login
2. AI Pre-assessment
3. Learning Path
4. Write Code
5. Compile & Run
6. AI Error Repair
7. Visualise
8. Earn XP & Badges
9. Interview Prep

VERIFIED IN SOURCE

- ✓ stderr alone is not treated as failure
- ✓ 11 JSON generators, each with a mock fallback
- ✓ TF-IDF RAG: ~900-char chunks, 150 overlap
- ✓ Failed quiz → adaptive remedial help
- ✓ 7 interview-prep endpoints, voice in/out

ANALYTICS: THE CODEBASE IN NUMBERS

Overall: lines of code



Frontend	22,459	78%
Backend	6,310	22%

Frontend: lines by area



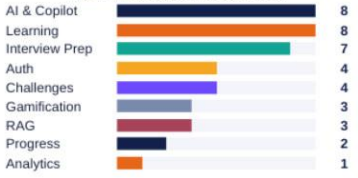
Components	11,208	50%
Pages (30)	9,138	41%
Hooks	550	2%
Shell, utils, types	1,563	7%

Backend: lines by layer



Services	1,942	31%
Controllers	1,425	23%
Models	407	6%
Utils	205	3%
Routes	149	2%
Middleware	38	1%
Server & scripts	2,144	34%

REST endpoints per module (40)



Counts are JS/TS/JSX lines in the repository, excluding node_modules, build output and temp folders. "Server, seed & scripts" covers server bootstrap, seed and debug scripts.

IMPLEMENTATION STATUS (VERIFIED IN SOURCE)



Working in live app	6	60%
Implemented, not wired	3	30%
Not yet built	1	10%

LIVE	Auth (JWT + bcrypt)	LIVE	Code execution engine
LIVE	Multi-provider LLM router	LIVE	TF-IDF document RAG
LIVE	Gamification engine	LIVE	Interview simulator
UNWIRED	Agent loop (planner-executor)	UNWIRED	Prompt-chain module
UNWIRED	Qdrant semantic search	PLANNED	Docker / container sandbox

LLM FALLBACK: WHO ANSWERS?

Gemini	Groq	Ollama	Answered by
✓ on	✓ on	✓ on	**Gemini** first try
✗ off	✓ on	✓ on	**Groq** after 1 failed attempt
✗ off	✗ off	✓ on	**Ollama (local)** after 2 failed attempts
✗ off	✗ off	✗ off	**Mock generator** valid canned content, no error

Two live model-deprecation incidents (Gemini and Groq model names) were absorbed by this chain, and masked for a time until a start-up diagnostic was added.

"FOUR AGENTS": DESIGN VS. IMPLEMENTATION

Agent	As designed	Implemented today
Code Analysis	Detects syntax/logic errors, suggests fixes	One JSON call (`analyzeCodeFeedback` / `debugCodeError`) via the shared router
Execution Visualization	Real-time visuals of variables, loops, structures	Client-side React visualisers; no server-side AI
Learning Assistant	Answers questions, personalised guidance	`generateChatReply` (+ stream) and TF-IDF document Q&A
Assessment	Evaluates skills, generates challenges	`generateSkillProfile`, interview quiz, `evaluateInterviewCode`

IMPLEMENTATION: WORKING PROTOTYPE

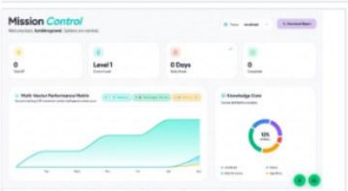


Fig. 1 Mission Control: XP, level, streak, performance chart and knowledge-core donut

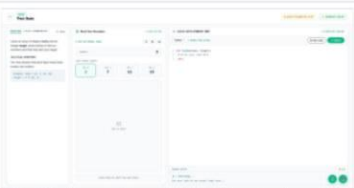


Fig. 2 Challenge workspace: brief, real-time visualiser, editor and console



Fig. 3 DSA visualiser: step-by-step sorting animation

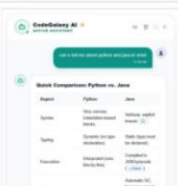


Fig. 4 AI assistant with text/voice input and Markdown tables

TECHNOLOGY STACK & GAMIFICATION

React 18 · TypeScript · Vite · Tailwind CSS · Radix UI · Monaco · React Flow · Recharts · Node.js · Express 5 · MongoDB Atlas · Mongoose 9 · JWT · bcrypt · Gemini · Groq · Ollama · pdf-parse · Vercel · Render

- ✓ XP, levels, streaks, badges + "solar-system" skill map
- ✓ Prerequisite-linked skill nodes; "Nebula Forge" AI challenge generator

CONCLUSION

CodeGalaxy is a working, full-stack programming-education platform with a real multi-language execution engine, a resilient three-provider LLM fallback chain, a lexical document-retrieval pipeline, a persistent gamification model and an AI interview simulator. Measured against its own design documents it is honestly AI-assisted rather than fully agentic today: a genuine planner-executor-synthesizer loop exists in code but is not yet reachable from any route.

LIMITATIONS & FUTURE WORK

Limitations: execution runs in a host child process (not container-isolated); document store is in-memory and single-instance; image Q&A depends on one Groq vision model; no automated test suite.

Future work: wire the agent loop into the repair endpoint; Docker / gVisor sandbox; Qdrant semantic search; SSE streaming; per-tenant rate limiting; offline-first sync.

Live demo: code-galaxy-an-agentic-ai-based-int.vercel.app

Source: github.com/HarshKumar5822/CodeGalaxy-An-Agentic-AI-Based-Interactive-Programming-Learning-Platform